# Tech Tools Cheatsheet

A Reference Guide to Technical Tools

Created by Jonathan Ma

October 10, 2026

# Contents

**1 Data Acquisition** **2**
1.1 REDCap . . . 2
1.2 FHIR . . . 6
1.3 Epic . . . 11
1.4 Bloomberg Terminal . . . 15

**2 Storage and Query** **20**
2.1 SQL . . . 20
2.2 DuckDB . . . 40
2.3 OMOP Common Data Model . . . 45

**3 Analysis and Modeling** **52**
3.1 Google Colab . . . 52
3.2 SAS . . . 54
3.3 Stata . . . 59
3.4 JAX . . . 64
3.5 Hugging Face . . . 68
3.6 LangChain . . . 74

**4 Validation** **80**
4.1 MLflow . . . 80

**5 Deployment** **93**
5.1 Docker . . . 93

**6 Reporting** **108**
6.1 Power BI . . . 108
6.2 Tableau . . . 117

# 1 Data Acquisition

## 1.1 REDCap

**REDCap (Research Electronic Data Capture)** is a secure platform for collecting and managing research data, particularly clinical and academic research data.

The basic hierarchy is:

$$\text{Project} \rightarrow \text{Record} \rightarrow \text{Instrument} \rightarrow \text{Field}$$

For longitudinal studies:

$$\text{Project} \rightarrow \text{Record} \rightarrow \text{Event} \rightarrow \text{Instrument} \rightarrow \text{Field}$$

**Core terminology**

| Term | Meaning |
|---|---|
| Project | A study, survey, registry, or research database |
| Record | One observational unit, usually a participant |
| Record ID | Unique identifier for a record |
| Instrument | A form containing related fields |
| Field | Individual variable collected in an instrument |
| Event | Study time point in a longitudinal project |
| Arm | Study pathway or group containing events |
| DAG | Data Access Group restricting records by site/group |
| Data Dictionary | CSV representation of fields and project structure |

**Common field types**

- Text
- Notes
- Multiple choice / radio buttons
- Dropdown
- Checkbox
- Yes/No
- True/False
- Calculated field
- File upload

Text fields can also use validation such as integer, number, date, datetime, email, and other formats.

### Variable names versus labels

A field has both a machine-friendly variable name and a human-readable label.

For example:

```
Variable: systolic_bp
Label:    Systolic Blood Pressure (mmHg)
```

Use stable, descriptive variable names. Avoid embedding information that may later change into variable names.

Prefer:

```
baseline_weight
treatment_group
visit_date
```

over ambiguous names such as:

```
var1
question2
data3
```

### Branching logic

Branching logic conditionally displays fields based on other responses.

```
[smoker] = "1"
```

Multiple conditions:

```
[smoker] = "1" and [age] >= 18
```

Alternative conditions:

```
[treatment] = "1" or [treatment] = "2"
```

For checkbox fields, individual choices are referenced separately:

```
[symptoms(1)] = "1"
```

### Calculated fields

Calculated fields can derive values from existing fields.

For example, BMI:

```
[weight_kg] / ([height_m] ^ 2)
```

Calculated fields are useful for simple deterministic quantities, but complex analysis is generally better performed outside the data-entry layer.

### Longitudinal projects

Longitudinal projects associate instruments with defined events such as:

```
Baseline
Week 4
Week 8
Month 6
```

The same variable can therefore be observed at different study time points.

An event-specific reference can look like:

```
[baseline_arm_1][weight_kg]
```

### Repeating instruments

Repeating instruments allow multiple instances of the same form for a record, such as repeated:

- adverse events;
- medications;
- hospital visits;
- follow-up assessments.

Do not confuse **repeating instruments** with **longitudinal events**. Events represent planned study time points; repeating instruments represent an arbitrary number of repeated instances.

### Data Dictionary

The Data Dictionary is one of the most useful REDCap tools for larger projects. It allows project fields to be inspected or defined in spreadsheet form.

Important information includes:

- variable name;
- instrument/form name;
- field type;
- field label;
- choices/codings;
- validation;
- calculation;
- branching logic;

- required-field status.

For large projects, editing a Data Dictionary can be substantially faster and more reproducible than manually creating every field through the interface.

**Coded values**

Always distinguish between the **stored value** and the **displayed label**.

For example:

```
0, Control
1, Treatment
```

The database may store:

```
1
```

while the user sees:

```
Treatment
```

This distinction matters when exporting and analyzing REDCap data.

**Checkboxes**

Checkboxes deserve particular attention because multiple choices can be selected simultaneously. Exports may represent each option as a separate indicator variable rather than one categorical variable.

Conceptually:

```
symptoms___1
symptoms___2
symptoms___3
```

These may correspond to separate yes/no indicators for each response option.

**Data quality**

Useful REDCap design practices include:

- validate values at entry when possible;
- use required fields selectively;
- define sensible minimum/maximum ranges;
- use branching logic to prevent irrelevant questions;
- avoid unnecessary free-text fields;
- use standardized coding consistently;

- test instruments before production use.

**Practical tips**

- Build a small prototype before constructing a large project.
- Decide variable naming conventions before adding hundreds of fields.
- Keep a copy of the Data Dictionary for reproducibility.
- Check whether exports use raw coded values or displayed labels.
- Be especially careful with checkbox exports and repeating instruments.
- Test branching logic using several realistic records.
- Avoid changing variable meaning after data collection has started.
- When possible, constrain data during collection instead of cleaning preventable errors afterward.

**Interview notes**

Be able to explain the distinction between a record, instrument, field, event, and repeating instrument. A useful design principle is that REDCap should enforce as much valid structure as practical *during collection*, because preventing invalid data is generally preferable to repairing it downstream.

## 1.2 FHIR

**FHIR (Fast Healthcare Interoperability Resources)** is an HL7 standard for representing and exchanging healthcare information.

FHIR organizes healthcare information into modular **resources**.

**Common resources**

| Resource | Represents |
|---|---|
| Patient | Demographics and patient identity |
| Encounter | Interaction with a healthcare system |
| Condition | Diagnosis or clinical condition |
| Observation | Measurement, lab result, or vital sign |
| Procedure | Clinical procedure |
| Medication | Medication definition |
| MedicationRequest | Medication order or prescription |
| DiagnosticReport | Diagnostic/laboratory report |
| Practitioner | Healthcare professional |
| Organization | Healthcare organization |
| AllergyIntolerance | Allergy or intolerance |
| Immunization | Vaccination event |

A useful mental model is:

$$\text{Patient} \longleftarrow \begin{cases} \text{Encounter} \\ \text{Observation} \\ \text{Condition} \\ \text{Procedure} \\ \text{MedicationRequest} \end{cases}$$

FHIR therefore does *not* generally represent an entire patient's history as one giant record.

**Resource structure**

FHIR resources are commonly represented using JSON or XML.

A simplified Patient resource:

```
{
  "resourceType": "Patient",
  "id": "123",
  "name": [
    {
      "family": "Smith",
      "given": ["Jane"]
    }
  ],
  "gender": "female",
  "birthDate": "1985-04-12"
}
```

The `resourceType` identifies what kind of resource the object represents.

**References**

FHIR resources are connected through references.

An Observation might contain:

```
"subject": {
  "reference": "Patient/123"
}
```

An Encounter might similarly reference the patient:

```
"subject": {
  "reference": "Patient/123"
}
```

This means that interpreting clinical data often requires understanding relationships among several resources.

### Observation

Observation is one of the most important resources for data work.

A simplified blood-pressure observation might contain:

```
{
  "resourceType": "Observation",
  "status": "final",
  "code": {
    "coding": [
      {
        "system": "http://loinc.org",
        "code": "85354-9"
      }
    ]
  },
  "subject": {
    "reference": "Patient/123"
  }
}
```

An Observation is not synonymous with a laboratory result. It can represent many measured clinical facts, including:

- laboratory measurements;
- vital signs;
- body measurements;
- clinical scores;
- other measured observations.

### Clinical coding systems

FHIR resources frequently contain codes from external clinical terminology systems.

| System | Typical Use |
|---|---|
| LOINC | Laboratory tests and clinical observations |
| SNOMED CT | Detailed clinical terminology |
| ICD-10 | Diagnosis classification |
| RxNorm | Medication terminology |
| CPT | Procedures/services, particularly U.S. billing contexts |

A coded element often contains:

```
{
  "system": "http://loinc.org",
  "code": "8480-6",
  "display": "Systolic blood pressure"
}
```

Keep the distinction clear:

$$\text{FHIR} = \text{structure/interoperability}$$

while systems such as LOINC and SNOMED CT provide much of the clinical terminology.

**CodeableConcept**

FHIR frequently represents coded clinical information using a `CodeableConcept`.

A concept can contain multiple codings:

```
"code": {
  "coding": [
    {
      "system": "...",
      "code": "...",
      "display": "..."
    }
  ],
  "text": "Human-readable description"
}
```

Do not assume the human-readable `display` string is the canonical identifier. The coding system and code carry the standardized meaning.

**Cardinality**

FHIR specifications use cardinalities such as:

```
0..1
1..1
0..*
1..*
```

Interpretation:

| Cardinality | Meaning |
|---|---|
| `0..1` | Optional, at most one |
| `1..1` | Required, exactly one |
| `0..*` | Optional, any number |
| `1..*` | At least one |

This is useful when reading FHIR specifications and understanding why some fields are arrays while others are singular values.

**Profiles**

FHIR itself is flexible. A **profile** constrains a base resource for a particular use case.

A profile may:

- require fields that are optional in base FHIR;
- restrict allowable terminology;
- constrain cardinality;
- define expected extensions.

Therefore, "FHIR compliant" does not imply that every implementation stores identical fields in identical ways.

**Extensions**

FHIR supports extensions for information not represented directly by the base resource.

Conceptually:

```
"extension": [
  {
    "url": "...",
    "valueString": "..."
  }
]
```

Extensions are an intentional part of FHIR's design, not necessarily a violation of the standard.

**Bundles**

Collections of resources can be represented as a **Bundle**.

```
{
  "resourceType": "Bundle",
  "type": "collection",
  "entry": [
    {
      "resource": { ... }
```

```
    },
    {
      "resource": { ... }
    }
  ]
}
```

The important practical point is that a Bundle contains multiple resource entries; it is not itself a Patient, Observation, or Encounter.

**Practical tips**

- Always inspect `resourceType` before assuming an object's structure.
- Expect nested objects and arrays rather than flat tabular data.
- Do not identify clinical concepts using display text alone.
- Preserve coding-system information along with clinical codes.
- Expect resources to reference other resources.
- Check the FHIR version and implementation profile being used.
- Do not assume an optional field will be populated simply because the standard defines it.
- Be careful when flattening FHIR into tables; one-to-many relationships can easily duplicate patient-level rows.

**Interview notes**

The most important distinction is:

FHIR is an interoperability standard, not an EHR or analytics schema.

Be comfortable explaining resources, references, terminology systems, profiles, extensions, and cardinality. Also understand why converting nested FHIR resources into analytical tables is not always a simple one-row-per-patient operation.

## 1.3 Epic

**Epic** is a major Electronic Health Record (EHR) platform used for clinical documentation, orders, results, scheduling, billing, and other healthcare workflows.

For technical work, the most important point is that "Epic" refers to an ecosystem rather than a single flat database.

**Useful conceptual layers**

| Component | Primary Role |
|---|---|
| Epic EHR | Operational clinical system |
| Hyperspace | Traditional Epic end-user application |
| Hyperdrive | Web-based Epic client environment |
| Reporting Workbench | Operational reporting |
| Clarity | Relational reporting database |
| Caboodle | Enterprise analytics/data warehouse |

A useful mental model is:

$$\text{Clinical Workflow} \to \text{Operational Epic Data} \to \begin{cases} \text{Operational Reporting} \\ \text{Clarity} \\ \text{Caboodle} \end{cases}$$

**Operational versus analytical data**

The live EHR is optimized for clinical operations:

- recording encounters;
- documenting clinical notes;
- entering orders;
- recording results;
- scheduling;
- billing and administrative workflows.

Large analytical workloads generally rely on downstream reporting or warehouse systems rather than treating the live operational system as a general-purpose analytics database.

**Clarity**

**Clarity** is Epic's relational reporting database. Operational Epic data is transformed into relational tables suitable for reporting, research, and analysis.

Conceptually, analysts may encounter data corresponding to:

- patients;
- encounters;
- diagnoses;
- medications;
- orders;

- procedures;
- laboratory results;
- providers.

Clarity is important because EHR data is naturally relational. A patient can have many encounters, and each encounter can have many diagnoses, orders, medications, or results.

$$\text{Patient} \to \text{Encounter} \to \begin{cases} \text{Diagnosis} \\ \text{Order} \\ \text{Medication} \\ \text{Result} \end{cases}$$

### Caboodle

**Caboodle** is Epic's enterprise data warehouse. It provides a more analytics-oriented representation of data from Epic and potentially other organizational systems.

A useful distinction is:

$$\text{Clarity} \approx \text{detailed relational reporting data}$$

$$\text{Caboodle} \approx \text{enterprise analytics warehouse}$$

Exact implementations and available data vary across organizations.

### Reporting Workbench

Reporting Workbench supports operational reporting closer to day-to-day EHR workflows. It is useful for operational lists and reports but serves a different purpose from a large analytical warehouse.

### Encounter-centric data

One of the most important concepts when working with EHR data is that **patient** and **encounter** are different units.

A patient may have:

$$1 \text{ patient} \to N \text{ encounters}$$

and one encounter may have:

$$1 \text{ encounter} \rightarrow N \text{ diagnoses/results/orders}$$

Joining these tables without understanding their grain can dramatically multiply rows.

Before using an EHR dataset, ask:

> What does one row represent?

Possible answers include:

- one patient;
- one encounter;
- one diagnosis;
- one medication order;
- one lab result;
- one procedure.

**Dates and timestamps**

Clinical data can contain several timestamps describing different events.

For example, a laboratory record might involve:

$$\text{Order Time} \rightarrow \text{Collection Time} \rightarrow \text{Result Time}$$

These are not interchangeable.

Similarly, an encounter may have scheduled, arrival, admission, discharge, and documentation timestamps.

Always determine which timestamp corresponds to the scientific or operational question.

**Clinical codes**

EHR data can contain both standardized and organization-specific codes.

Do not assume that:

- every diagnosis is represented identically across institutions;
- every local laboratory code maps trivially to one standard code;
- displayed names are stable identifiers;
- absence of a code necessarily means absence of the condition.

**Practical tips**

- Determine the grain of every table before joining.
- Check key uniqueness before assuming a one-to-one relationship.
- Distinguish patient-level identifiers from encounter-level identifiers.
- Inspect date semantics rather than selecting whichever timestamp is easiest.
- Expect repeated measurements and repeated encounters.
- Treat missing clinical data carefully; "not recorded" is not always equivalent to "did not occur."
- Preserve provenance when combining data from multiple clinical sources.
- Remember that Epic implementations differ across healthcare organizations.

**Interview notes**

Be able to explain why operational EHR systems and analytical warehouses have different purposes. For data roles, emphasize **grain, temporal semantics, join cardinality, provenance, and clinical coding**. A technically correct join can still produce a clinically incorrect dataset if these are not understood.

## 1.4 Bloomberg Terminal

**Bloomberg Terminal** is a professional financial information platform used for market data, company fundamentals, economic data, news, screening, valuation, and financial analysis.

Bloomberg workflows rely heavily on short mnemonic commands followed by the `<GO>` key.

**Security syntax**

A security is commonly specified using:

$$\text{Identifier} + \text{Market/Asset Type}$$

Examples:

```
AAPL US Equity <GO>
NVDA US Equity <GO>
SPX Index <GO>
EURUSD Curncy <GO>
```

Then apply a function:

```
AAPL US Equity DES <GO>
AAPL US Equity GP <GO>
AAPL US Equity FA <GO>
```

**Useful everyday functions**

| Function | Purpose |
|---|---|
| DES | Security description / overview |
| GP | Price chart |
| HP | Historical prices |
| FA | Financial analysis |
| DVD | Dividend information |
| CN | Company news |
| ANR | Analyst recommendations |
| ERN | Earnings information |
| RV | Relative valuation |
| EQS | Equity screening |
| WEI | Global equity indices |
| ECST | Economic statistics |
| TOP | Top Bloomberg news |
| HELP | Context-sensitive help |

A good practical habit is to use:

```
DES <GO>
```

when unfamiliar with a security, then move into more specialized functions.

**Asset-class identifiers**

Common Bloomberg security types include:

| Suffix | Asset Type |
|---|---|
| Equity | Equity security |
| Index | Index/rate series |
| Curncy | Currency |
| Corp | Corporate bond |
| Govt | Government security |
| Comdty | Commodity |

Do not assume a ticker alone uniquely identifies a financial instrument.

**Excel Add-In**

The Bloomberg Excel Add-In is extremely useful for bringing Terminal data into spreadsheet workflows.

Three functions are especially important:

`BDP, BDH, BDS`

### BDP: Bloomberg Data Point

Retrieve a current/reference value:

```
=BDP("AAPL US Equity","PX_LAST")
```

Examples:

```
=BDP("AAPL US Equity","CUR_MKT_CAP")
=BDP("AAPL US Equity","PE_RATIO")
```

### BDH: Bloomberg Data History

Retrieve historical time-series data:

```
=BDH(
    "AAPL US Equity",
    "PX_LAST",
    "01/01/2026",
    "10/01/2026"
)
```

### BDS: Bloomberg Data Set

Retrieve bulk/multi-row information:

```
=BDS(
    "AAPL US Equity",
    "TOP_20_HOLDERS_PUBLIC_FILINGS"
)
```

Remember:

| Function | Use |
|---|---|
| BDP | One/reference data point |
| BDH | Historical time series |
| BDS | Bulk/multi-row data |

### Common fields

Bloomberg data fields use mnemonics such as:

```
PX_LAST
PX_OPEN
PX_HIGH
PX_LOW
VOLUME
CUR_MKT_CAP
PE_RATIO
```

When unsure of a field, use Bloomberg's field search rather than guessing the mnemonic.

**Historical data**

Before using downloaded financial time series, verify:

- observation frequency;
- date range;
- currency;
- trading calendar;
- missing/non-trading dates;
- corporate-action treatment;
- adjusted versus unadjusted values.

**Corporate actions**

Stock splits, dividends, spin-offs, and other corporate actions can affect historical prices and returns.

For example, a raw price discontinuity caused by a stock split is not an economic return:

$$P_t \ll P_{t-1}$$

may reflect a mechanical adjustment rather than a market loss.

Always know whether the requested series is adjusted appropriately for the analysis.

**Point-in-time data**

For historical modeling and backtesting, distinguish:

$$\text{Current Database Value} \neq \text{Value Known at Historical Date}$$

Financial databases may contain revised fundamentals, classifications, or other information that was not available to market participants at the time.

Using current information in a historical model creates **look-ahead bias**.

Examples of potential problems include:

- revised economic data;
- restated financial statements;
- current index constituents;
- current industry classifications;

- later analyst estimates.

**Survivorship bias**

A current list of securities excludes firms that disappeared through bankruptcy, acquisition, delisting, or other events.

Backtesting only currently surviving firms can therefore produce overly optimistic results.

**Frequency alignment**

When combining Bloomberg series, frequencies may differ:

$$\text{Daily Equity Prices} + \text{Monthly CPI} + \text{Quarterly Earnings}$$

Do not mechanically forward-fill or merge series without considering when information actually became available.

For macroeconomic data in particular:

$$\text{Reference Period} \neq \text{Release Date}$$

A value describing September may not have been known until October.

**Practical tips**

- Start with `DES` when exploring an unfamiliar security.
- Remember `BDP = point`, `BDH = history`, `BDS = dataset`.
- Confirm security type when tickers are ambiguous.
- Verify units and currencies before comparing securities.
- Check whether historical prices account for corporate actions.
- Record the field mnemonic used to make an extraction reproducible.
- For backtests, ask whether the data are truly point-in-time.
- Check index membership historically rather than using only current constituents.
- Treat release dates carefully when combining market and macroeconomic data.

**Interview notes**

For finance/data interviews, the most important practical issues are often not how to click through Bloomberg, but whether you understand the data being retrieved. Be ready to discuss **adjusted prices, corporate actions, point-in-time data, look-ahead bias, survivorship bias, security identifiers, and frequency alignment**.

For Bloomberg-specific usage, know the distinction between `BDP`, `BDH`, and `BDS`, and recognize common Terminal functions such as `DES`, `GP`, `FA`, `EQS`, and `ECST`.

# 2 Storage and Query

## 2.1 SQL

**SQL (Structured Query Language)** is the standard language for working with relational databases. For day-to-day analytics, the most important skills are selecting and filtering data, joining tables, aggregating observations, using window functions, and understanding the grain of a query result.

A useful mental model for a SQL query is:

$$\texttt{FROM} \to \texttt{WHERE} \to \texttt{GROUP BY} \to \texttt{HAVING} \to \texttt{SELECT} \to \texttt{ORDER BY} \to \texttt{LIMIT}$$

This is the approximate *logical* processing order, not necessarily the physical execution order chosen by the database optimizer.

### Core SQL

#### Basic query

```
SELECT
    customer_id,
    revenue
FROM customers
WHERE revenue > 1000
ORDER BY revenue DESC
LIMIT 10;
```

Avoid using:

```
SELECT *
```

in production queries when only a few columns are required. Explicit columns make the expected schema clearer and can reduce unnecessary data reads.

#### Filtering

```
SELECT *
FROM employees
WHERE salary >= 80000
  AND department = 'Analytics';
```

Common conditions:

```
WHERE x = 10
WHERE x <> 10
WHERE x > 10
WHERE x BETWEEN 10 AND 20
WHERE x IN (10, 20, 30)
WHERE name LIKE 'Jon%'
WHERE name LIKE '%Smith%'
WHERE x IS NULL
WHERE x IS NOT NULL
```

### NULL

`NULL` represents an unknown or missing value.

Do not use:

```
WHERE x = NULL
```

Use:

```
WHERE x IS NULL
```

Similarly:

```
WHERE x IS NOT NULL
```

SQL uses three-valued logic:

$$\texttt{TRUE}, \quad \texttt{FALSE}, \quad \texttt{UNKNOWN}$$

Comparisons involving `NULL` generally evaluate to `UNKNOWN`.

### COALESCE

Return the first non-null value:

```
SELECT
    customer_id,
    COALESCE(phone, mobile_phone, 'Unknown') AS phone
FROM customers;
```

Replace missing numeric values:

```
SELECT
    COALESCE(revenue, 0) AS revenue
FROM customers;
```

Be careful: replacing missing values with zero changes their meaning and should only be done when zero is scientifically or operationally appropriate.

### CASE

```
SELECT
    customer_id,
    revenue,
    CASE
        WHEN revenue >= 10000 THEN 'High'
        WHEN revenue >= 5000 THEN 'Medium'
        ELSE 'Low'
    END AS revenue_group
FROM customers;
```

Conditional indicators are also common:

```
CASE
    WHEN age >= 18 THEN 1
    ELSE 0
END AS adult
```

### Aggregation

Common aggregate functions:

```
COUNT(*)
COUNT(column_name)
COUNT(DISTINCT column_name)

SUM(x)
AVG(x)
MIN(x)
MAX(x)
```

Example:

```
SELECT
    department,
    COUNT(*) AS n,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

A crucial distinction:

```
COUNT(*)
```

counts rows, while:

```
COUNT(column_name)
```

counts non-null values in that column.

### WHERE versus HAVING

`WHERE` filters rows before aggregation:

```
SELECT
    department,
    AVG(salary) AS avg_salary
FROM employees
WHERE active = 1
GROUP BY department;
```

`HAVING` filters groups after aggregation:

```
SELECT
    department,
    AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 90000;
```

A useful rule is:

`WHERE` = filter observations

`HAVING` = filter aggregated groups

### Joins

| Join | Result |
|---|---|
| INNER JOIN | Only rows with matches in both tables |
| LEFT JOIN | Every left-table row plus matching right-table data |
| RIGHT JOIN | Every right-table row plus matching left-table data |
| FULL OUTER JOIN | All rows from both tables |
| CROSS JOIN | Cartesian product of both tables |

Example:

```
SELECT
    c.customer_id,
    c.name,
    o.order_id,
```

```
    o.amount
FROM customers AS c
LEFT JOIN orders AS o
    ON c.customer_id = o.customer_id;
```

The most important practical question before joining is:

> What is the relationship between these keys?

Possible relationships are:

$$1:1, \qquad 1:N, \qquad N:1, \qquad N:M$$

If one customer has five orders, joining one customer row to the order table creates five output rows.

Before trusting a join, check key uniqueness.

For example:

```
SELECT
    customer_id,
    COUNT(*) AS n
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

If this returns rows, `customer_id` is not unique in that table.

**Duplicate rows after joins**

Suppose:

$$\text{Patient} \to 5 \text{ diagnoses}$$

and:

$$\text{Patient} \to 10 \text{ medications}$$

Joining both directly by patient can potentially produce:

$$5 \times 10 = 50$$

rows for the patient.

This is a common real-world source of incorrect analyses.

### Subqueries

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

Subqueries can also appear in `FROM` or be used with operators such as `EXISTS`.

### EXISTS

```
SELECT *
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.customer_id
);
```

This asks whether at least one matching order exists without requiring the matching rows to be returned.

### Common Table Expressions

CTEs make multi-stage queries easier to read.

```
WITH customer_totals AS (
    SELECT
        customer_id,
        SUM(amount) AS total_spend
    FROM orders
    GROUP BY customer_id
)

SELECT *
FROM customer_totals
WHERE total_spend > 10000;
```

Multiple CTEs:

```
WITH recent_orders AS (
    SELECT *
    FROM orders
    WHERE order_date >= '2026-01-01'
),
customer_totals AS (
    SELECT
```

```
        customer_id,
        SUM(amount) AS total_spend
    FROM recent_orders
    GROUP BY customer_id
)

SELECT *
FROM customer_totals;
```

CTEs are especially useful for breaking complicated transformations into logical stages.

### Window functions

Window functions calculate values across related rows without collapsing the result to one row per group.

General structure:

```
function(...) OVER (
    PARTITION BY ...
    ORDER BY ...
)
```

For example:

```
SELECT
    employee_id,
    department,
    salary,
    AVG(salary) OVER (
        PARTITION BY department
    ) AS department_avg
FROM employees;
```

Unlike `GROUP BY`, every employee remains in the output.

### ROW_NUMBER, RANK, and DENSE_RANK

```
SELECT
    employee_id,
    department,
    salary,
    ROW_NUMBER() OVER (
        PARTITION BY department
        ORDER BY salary DESC
    ) AS rn
FROM employees;
```

Differences:

| Function | Behavior |
|---|---|
| ROW_NUMBER() | Unique sequential number for every row |
| RANK() | Ties share a rank and subsequent ranks contain gaps |
| DENSE_RANK() | Ties share a rank without subsequent gaps |

If salaries are:

```
100
100
90
```

then:

| Salary | ROW_NUMBER | RANK | DENSE_RANK |
|---|---|---|---|
| 100 | 1 | 1 | 1 |
| 100 | 2 | 1 | 1 |
| 90 | 3 | 3 | 2 |

### Top $N$ per group

A very common interview pattern:

```
WITH ranked AS (
    SELECT
        employee_id,
        department,
        salary,
        ROW_NUMBER() OVER (
            PARTITION BY department
            ORDER BY salary DESC
        ) AS rn
    FROM employees
)

SELECT *
FROM ranked
WHERE rn <= 3;
```

### LAG and LEAD

Access previous or subsequent rows:

```
SELECT
    date,
    price,
    LAG(price) OVER (
        ORDER BY date
    ) AS previous_price
```

```
FROM prices;
```

Compute a change:

```
WITH lagged AS (
    SELECT
        date,
        price,
        LAG(price) OVER (
            ORDER BY date
        ) AS previous_price
    FROM prices
)

SELECT
    date,
    price,
    price - previous_price AS price_change
FROM lagged;
```

### Running totals

```
SELECT
    date,
    revenue,
    SUM(revenue) OVER (
        ORDER BY date
        ROWS BETWEEN UNBOUNDED PRECEDING
                 AND CURRENT ROW
    ) AS cumulative_revenue
FROM daily_sales;
```

### Deduplication

Find duplicate keys:

```
SELECT
    customer_id,
    COUNT(*) AS n
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Keep the most recent row per customer:

```
WITH ranked AS (
    SELECT
        *,
```

```
        ROW_NUMBER() OVER (
            PARTITION BY customer_id
            ORDER BY updated_at DESC
        ) AS rn
    FROM customers
)

SELECT *
FROM ranked
WHERE rn = 1;
```

### Set operations

```
SELECT customer_id
FROM online_customers

UNION

SELECT customer_id
FROM store_customers;
```

Important operations:

- `UNION`: combines results and removes duplicates;
- `UNION ALL`: combines results and preserves duplicates;
- `INTERSECT`: rows appearing in both queries;
- `EXCEPT`: rows in the first query but not the second.

Use `UNION ALL` when duplicate removal is unnecessary.

### Data modification

Insert:

```
INSERT INTO employees (
    employee_id,
    name,
    salary
)
VALUES (
    101,
    'Alice',
    95000
);
```

Update:

```
UPDATE employees
SET salary = 100000
WHERE employee_id = 101;
```

Delete:

```
DELETE FROM employees
WHERE employee_id = 101;
```

Always double-check the `WHERE` clause before running `UPDATE` or `DELETE`.

A useful habit is to first run:

```
SELECT *
FROM employees
WHERE employee_id = 101;
```

and verify the affected rows before modifying them.

**Keys**

A **primary key** uniquely identifies a row.

A **foreign key** references a key in another table.

For example:

$$\texttt{customers.customer_id} \longleftarrow \texttt{orders.customer_id}$$

Primary and foreign keys encode relationships between tables.

**Indexes**

Indexes can accelerate lookups, joins, and sorting.

Conceptually:

```
CREATE INDEX idx_customer_id
ON orders(customer_id);
```

Indexes are particularly useful for frequently searched or joined columns, but they have costs:

- additional storage;
- additional work during inserts and updates;
- not every query benefits from an index.

**Transactions**

Transactions group related modifications.

```
BEGIN;

UPDATE accounts
SET balance = balance - 100
WHERE account_id = 1;

UPDATE accounts
SET balance = balance + 100
WHERE account_id = 2;

COMMIT;
```

If something fails:

```
ROLLBACK;
```

Transactions are commonly described by **ACID**:

- **Atomicity**: the transaction succeeds as a unit or fails as a unit.
- **Consistency**: valid database rules remain satisfied.
- **Isolation**: concurrent transactions do not improperly interfere.
- **Durability**: committed changes persist.

**Practical tips**

- Always know what one row represents in every table.
- Check key uniqueness before joining.
- Compare row counts before and after important joins.
- Use CTEs to make complex transformations readable.
- Prefer explicit column names over `SELECT *`.
- Handle `NULL` intentionally rather than accidentally.
- Use `UNION ALL` when deduplication is unnecessary.
- Filter early when practical to avoid processing unnecessary rows.
- Verify the rows affected before running `UPDATE` or `DELETE`.
- Remember that date, string, and limit syntax varies somewhat by SQL dialect.

**Interview notes**

SQL interviews frequently test joins, aggregation, CTEs, window functions, deduplication, top-$N$-per-group problems, and `NULL` behavior.

Be especially comfortable explaining:

- `WHERE` versus `HAVING`;
- `INNER JOIN` versus `LEFT JOIN`;
- `COUNT(*)` versus `COUNT(column)`;
- `UNION` versus `UNION ALL`;
- `ROW_NUMBER` versus `RANK` versus `DENSE_RANK`;
- `GROUP BY` versus window functions;
- why joins can unexpectedly multiply rows;
- primary keys versus foreign keys;
- the basic purpose of indexes and transactions.

### Python Integration

Python commonly interacts with relational databases through a database driver or a database abstraction such as SQLAlchemy. For analytics, pandas is also commonly used to move query results into DataFrames.

#### Basic connection pattern

Using SQLite:

```
import sqlite3

conn = sqlite3.connect("database.db")

cursor = conn.cursor()

cursor.execute("""
    SELECT customer_id, revenue
    FROM customers
    WHERE revenue > ?
""", (1000,))

rows = cursor.fetchall()

cursor.close()
conn.close()
```

Common cursor operations:

```
cursor.execute(query)

cursor.fetchone()
```

```
cursor.fetchmany(100)
cursor.fetchall()
```

### Context managers

When supported, use context managers to manage connections safely:

```
import sqlite3

with sqlite3.connect("database.db") as conn:
    rows = conn.execute(
        "SELECT * FROM customers"
    ).fetchall()
```

### Parameterized queries

Do not create SQL queries using untrusted values through string interpolation.

Avoid:

```
query = f"""
SELECT *
FROM users
WHERE username = '{username}'
"""
```

Instead use parameters:

```
cursor.execute(
    """
    SELECT *
    FROM users
    WHERE username = ?
    """,
    (username,)
)
```

Parameterized queries correctly handle values and protect against SQL injection.

Placeholder syntax differs across database drivers.

### pandas

Read query results directly into a DataFrame:

```
import pandas as pd

df = pd.read_sql_query(
    """
    SELECT
```

```
        customer_id,
        revenue
    FROM customers
    WHERE revenue > 1000
    """,
    conn
)
```

Write a DataFrame:

```
df.to_sql(
    "results",
    conn,
    if_exists="append",
    index=False
)
```

Common `if_exists` options:

| Option | Behavior |
|---|---|
| `fail` | Raise an error if table exists |
| `replace` | Replace existing table |
| `append` | Add rows to existing table |

Be particularly careful with:

```
if_exists="replace"
```

because replacing a table can destroy the existing table and may also lose database-specific schema details.

### Push computation to SQL

Avoid:

```
df = pd.read_sql(
    "SELECT * FROM transactions",
    conn
)

df = df[df["year"] == 2026]
```

when the database can perform the filtering:

```
df = pd.read_sql(
    """
    SELECT *
    FROM transactions
```

```
    WHERE year = 2026
    """,
    conn
)
```

A useful principle is:

> Move computation to the data before moving data to Python.

This is especially important for large databases.

### Chunked reads

For large query results:

```
for chunk in pd.read_sql_query(
    query,
    conn,
    chunksize=10000
):
    process(chunk)
```

This prevents the entire result from being loaded into memory simultaneously.

### SQLAlchemy

SQLAlchemy provides a common database interface.

```
from sqlalchemy import create_engine

engine = create_engine(
    "sqlite:///database.db"
)
```

Use it with pandas:

```
df = pd.read_sql(
    "SELECT * FROM customers",
    engine
)
```

Parameterized SQL:

```
from sqlalchemy import text

query = text("""
    SELECT *
    FROM customers
```

```
    WHERE revenue > :min_revenue
""")

with engine.connect() as conn:
    result = conn.execute(
        query,
        {"min_revenue": 1000}
    )
```

### Transactions

```
with engine.begin() as conn:
    conn.execute(statement_1)
    conn.execute(statement_2)
```

The transaction commits when the block succeeds and rolls back if an exception occurs.

### Practical tips

- Use parameterized queries rather than string interpolation.
- Push filters, joins, and aggregation into SQL when practical.
- Do not load millions of unnecessary rows into pandas.
- Use chunking when results do not comfortably fit in memory.
- Close connections and cursors when they are no longer needed.
- Be cautious with `to_sql(..., if_exists="replace")`.
- Keep database credentials out of source code.
- Remember that pandas types and database types do not always map perfectly.

### Interview notes

Understand the division of labor:

$$\text{SQL} = \text{filter/join/aggregate near the data}$$

$$\text{Python} = \text{analysis/modeling/application logic}$$

A common interview question is why filtering in SQL before loading into pandas is preferable to retrieving an entire large table. The answer is reduced data transfer, lower memory use, and use of the database engine's query optimizer.

### R Integration

R commonly accesses relational databases through the `DBI` interface and a database-specific backend.

The typical workflow is:

$$\text{Connect} \rightarrow \text{Query} \rightarrow \text{Analyze} \rightarrow \text{Disconnect}$$

### Basic DBI workflow

Using SQLite:

```
library(DBI)
library(RSQLite)

con <- dbConnect(
    SQLite(),
    "database.db"
)

df <- dbGetQuery(
    con,
    "SELECT * FROM customers"
)

dbDisconnect(con)
```

### Useful DBI commands

```
dbListTables(con)

dbListFields(
    con,
    "customers"
)

dbExistsTable(
    con,
    "customers"
)

dbReadTable(
    con,
    "customers"
)

dbGetQuery(
```

```
    con,
    "SELECT * FROM customers"
)

dbDisconnect(con)
```

### dbGetQuery versus dbExecute

Use `dbGetQuery()` when SQL returns rows:

```
df <- dbGetQuery(
    con,
    "
    SELECT
        customer_id,
        revenue
    FROM customers
    "
)
```

Use `dbExecute()` for operations that modify data or schema:

```
dbExecute(
    con,
    "
    DELETE FROM customers
    WHERE inactive = 1
    "
)
```

### dbplyr

`dbplyr` allows many familiar `dplyr` operations to be translated into SQL.

```
library(dplyr)

customers <- tbl(
    con,
    "customers"
)

result <- customers |>
    filter(revenue > 1000) |>
    select(customer_id, revenue) |>
    arrange(desc(revenue))
```

A critical concept is that database-backed `dbplyr` operations are typically **lazy**.

The result has not necessarily been loaded into R.

Inspect the generated SQL:

```
result |>
    show_query()
```

Load the result into R:

```
df <- result |>
    collect()
```

Therefore:

$$\texttt{tbl()} \to \text{database-backed lazy query}$$

while:

$$\texttt{collect()} \to \text{materialized R object}$$

### Push computation to the database

Prefer:

```
result <- tbl(con, "transactions") |>
    filter(year == 2026) |>
    group_by(customer_id) |>
    summarize(
        total = sum(amount, na.rm = TRUE)
    ) |>
    collect()
```

over collecting the entire transaction table and then filtering it in R.

### Writing tables

```
dbWriteTable(
    con,
    "results",
    df,
    overwrite = TRUE
)
```

Append instead:

```
dbWriteTable(
    con,
    "results",
    df,
```

```
    append = TRUE
)
```

Use `overwrite = TRUE` carefully because it replaces the existing table.

**Practical tips**

- Use `show_query()` when unsure how `dbplyr` translates an operation.
- Delay `collect()` until the data have been reduced to a manageable size.
- Push filters, joins, and summaries to the database.
- Disconnect from the database when finished.
- Avoid hard-coding database credentials in scripts.
- Be cautious when overwriting existing tables.
- Remember that not every R function has a SQL translation.

**Interview notes**

The most important concept is **lazy evaluation**. With `dbplyr`, the R code often describes a query that executes in the database rather than immediately creating an in-memory R data frame.

Be comfortable explaining why:

```
collect()
```

should usually occur *after* filtering and aggregation rather than before.

## 2.2 DuckDB

**DuckDB** is an embedded analytical SQL database designed for fast OLAP-style workloads. It is particularly useful for querying local analytical data such as Parquet and CSV files without operating a separate database server.

A useful analogy is:

SQLite : embedded transactional database

DuckDB : embedded analytical database

**Why use DuckDB?**

DuckDB is useful when:

- data are larger than is convenient for ordinary DataFrame workflows;

- data are stored in many CSV or Parquet files;
- SQL is convenient for the analysis;
- running a database server would be unnecessary overhead;
- only a subset of a large dataset needs to be queried.

### CLI

Start an in-memory session:

```
duckdb
```

Open or create a persistent database:

```
duckdb analytics.duckdb
```

Useful commands:

```
.tables
.schema
.schema customers
.describe customers
.help
.quit
```

### Query CSV directly

```
SELECT *
FROM 'customers.csv'
LIMIT 10;
```

Explicit CSV reader:

```
SELECT *
FROM read_csv_auto('customers.csv');
```

Create a persistent table:

```
CREATE TABLE customers AS
SELECT *
FROM read_csv_auto('customers.csv');
```

### Query Parquet directly

```
SELECT *
FROM 'transactions.parquet'
LIMIT 10;
```

or:

```
SELECT *
FROM read_parquet('transactions.parquet');
```

Query many files:

```
SELECT *
FROM read_parquet('transactions/*.parquet');
```

Aggregate without importing the files first:

```
SELECT
    customer_id,
    SUM(amount) AS total_spend
FROM read_parquet('transactions/*.parquet')
WHERE transaction_date >= '2026-01-01'
GROUP BY customer_id;
```

This is one of DuckDB's most useful everyday capabilities.

### Views over files

Create a reusable view:

```
CREATE VIEW transactions AS
SELECT *
FROM read_parquet(
    'transactions/*.parquet'
);
```

Then:

```
SELECT
    customer_id,
    SUM(amount) AS total_spend
FROM transactions
GROUP BY customer_id;
```

### Export to Parquet

```
COPY (
    SELECT *
    FROM transactions
    WHERE transaction_date >= '2026-01-01'
)
TO 'transactions_2026.parquet'
(FORMAT PARQUET);
```

Export CSV:

```
COPY (
    SELECT *
    FROM transactions
)
TO 'transactions.csv'
(HEADER, DELIMITER ',');
```

### Parquet

Parquet is a **columnar** file format.

For an analytical query such as:

```
SELECT
    customer_id,
    amount
FROM read_parquet('large.parquet')
WHERE year = 2026;
```

DuckDB can often avoid reading irrelevant data.

Two important concepts are:

- **Projection pushdown**: read only required columns.
- **Predicate pushdown**: avoid reading data that cannot satisfy filters.

This is why DuckDB + Parquet can be substantially more efficient than loading an entire CSV into memory.

### EXPLAIN

Inspect the query plan:

```
EXPLAIN
SELECT
    customer_id,
    SUM(amount)
FROM transactions
GROUP BY customer_id;
```

Inspect execution information:

```
EXPLAIN ANALYZE
SELECT
    customer_id,
    SUM(amount)
FROM transactions
GROUP BY customer_id;
```

### DuckDB versus pandas

A useful rule of thumb is:

$$\text{pandas} \rightarrow \text{in-memory manipulation}$$

$$\text{DuckDB} \rightarrow \text{SQL analytical engine}$$

DuckDB is particularly attractive when only a small subset or aggregation of a large file collection is needed.

### DuckDB versus server databases

| | DuckDB | Server Database |
|---|---|---|
| Architecture | Embedded | Client/server |
| Setup | Minimal | Server infrastructure |
| Primary strength | Local analytics | Shared persistent workloads |
| Parquet querying | Excellent | System dependent |
| Multi-user workloads | Not primary focus | Common use case |

### Practical tips

- Prefer Parquet over CSV for repeated analytical work when possible.
- Query files directly before deciding whether they need to become tables.
- Use wildcards to query partitioned file collections.
- Select only the columns required by the analysis.
- Filter early to take advantage of predicate pushdown.
- Use `EXPLAIN ANALYZE` when investigating slow queries.
- Do not deploy DuckDB as though it were a traditional high-concurrency application database.

### Interview notes

The core distinction is:

$$\boxed{\text{DuckDB is an embedded OLAP database.}}$$

Be able to explain why columnar storage is useful for analytics, why Parquet pairs well with DuckDB, and what predicate and projection pushdown mean.

A useful comparison is that PostgreSQL/MySQL commonly operate as database servers, SQLite is an embedded database commonly associated with transactional application workloads, and DuckDB

is optimized for embedded analytical workloads.

## 2.3 OMOP Common Data Model

**OMOP (Observational Medical Outcomes Partnership) Common Data Model** is a standardized relational data model maintained by the OHDSI community for observational healthcare data.

Its purpose is to transform heterogeneous healthcare data into a common structure and vocabulary so that analyses can be reused across datasets and institutions.

A useful mental model is:

$$\text{EHR / Claims / Registry} \rightarrow \text{ETL} \rightarrow \text{OMOP CDM} \rightarrow \text{Standardized Analysis}$$

### FHIR versus OMOP

The distinction is important:

$$\boxed{\text{FHIR} \rightarrow \text{interoperability}}$$

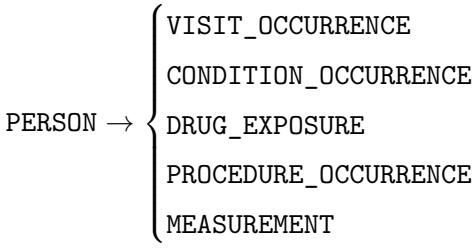
$$\boxed{\text{OMOP} \rightarrow \text{observational analytics}}$$

FHIR is resource-oriented and primarily designed for healthcare information exchange. OMOP is relational and primarily designed for standardized observational research and analytics.

**Core clinical tables**

| Table | Purpose |
|---|---|
| PERSON | Patient/person demographics |
| VISIT_OCCURRENCE | Healthcare encounters |
| CONDITION_OCCURRENCE | Diagnoses and conditions |
| DRUG_EXPOSURE | Medication exposures |
| PROCEDURE_OCCURRENCE | Procedures |
| MEASUREMENT | Labs and quantitative measurements |
| OBSERVATION | Clinical facts not represented elsewhere |
| DEVICE_EXPOSURE | Medical device exposure |
| SPECIMEN | Biological specimens |
| DEATH | Mortality information |

A simplified relationship is:




Most clinical event tables therefore contain a `person_id`.

### PERSON

The `PERSON` table contains one row per person.

Common concepts include:

```
person_id
gender_concept_id
year_of_birth
month_of_birth
day_of_birth
race_concept_id
ethnicity_concept_id
```

The exact availability of demographic fields depends on the source data.

### VISIT_OCCURRENCE

Visits represent healthcare encounters.

Conceptually:

```
person_id
visit_concept_id
visit_start_date
visit_end_date
```

Clinical events may reference the visit during which they occurred through `visit_occurrence_id`.

### CONDITION_OCCURRENCE

Conditions and diagnoses are represented as events:

```
SELECT
    person_id,
    condition_concept_id,
    condition_start_date
FROM condition_occurrence
WHERE person_id = 123;
```

### MEASUREMENT

Measurements commonly represent laboratory results and other quantitative clinical measurements.

Important concepts include:

```
person_id
measurement_concept_id
measurement_date
value_as_number
value_as_concept_id
unit_concept_id
```

When analyzing measurements, do not ignore the unit.

For example:

$$100 \text{ mg/dL} \neq 100 \text{ mmol/L}$$

#### Standard concepts

OMOP relies heavily on standardized concept identifiers.

Conceptually:

$$\text{Source Code} \rightarrow \text{Standard Concept}$$

Clinical tables often contain both standardized and source representations.

For example:

```
condition_concept_id
condition_source_concept_id
condition_source_value
```

The standardized concept enables portable analysis, while source fields preserve information from the original system.

### CONCEPT

The `CONCEPT` table provides information about vocabulary concepts.

Useful fields include:

```
concept_id
concept_name
domain_id
vocabulary_id
concept_class_id
```

```
standard_concept
concept_code
```

Join a condition to its human-readable concept:

```
SELECT
    c.person_id,
    c.condition_start_date,
    concept.concept_name
FROM condition_occurrence AS c
JOIN concept
    ON c.condition_concept_id =
       concept.concept_id
WHERE c.person_id = 123;
```

### Vocabulary tables

Important vocabulary-related tables include:

| Table | Purpose |
|---|---|
| CONCEPT | Individual standardized concepts |
| VOCABULARY | Vocabulary definitions |
| DOMAIN | Clinical concept domains |
| CONCEPT_RELATIONSHIP | Relationships among concepts |
| CONCEPT_ANCESTOR | Hierarchical ancestor/descendant relationships |
| CONCEPT_SYNONYM | Alternative concept names |

### Concept hierarchies

Suppose an analysis needs patients with any diagnosis belonging to a broader disease family.

Rather than manually listing every descendant code, `CONCEPT_ANCESTOR` can represent the hierarchy:

Broad Ancestor → Specific Descendants

For example:

```
SELECT
    descendant_concept_id
FROM concept_ancestor
WHERE ancestor_concept_id = 123456;
```

Use this to identify conditions belonging to the broader concept:

```
SELECT DISTINCT
    c.person_id
```

```
FROM condition_occurrence AS c
JOIN concept_ancestor AS ca
    ON c.condition_concept_id =
       ca.descendant_concept_id
WHERE ca.ancestor_concept_id = 123456;
```

### Domains

Standard concepts belong to domains such as:

- Condition;
- Drug;
- Procedure;
- Measurement;
- Observation;
- Device.

The domain helps determine where a standardized clinical concept belongs in the OMOP structure.

### Cohort construction

A common OMOP task is defining a patient cohort.

For example:

```
SELECT DISTINCT
    person_id
FROM condition_occurrence
WHERE condition_concept_id IN (
    12345,
    67890
);
```

Add a temporal criterion:

```
SELECT DISTINCT
    person_id
FROM condition_occurrence
WHERE condition_concept_id = 12345
  AND condition_start_date >= '2026-01-01';
```

Real cohort definitions may combine:

- diagnoses;
- medications;

- measurements;
- procedures;
- demographics;
- temporal relationships;
- inclusion/exclusion windows.

**Grain matters**

Different OMOP tables have different grains.

For example:

$$\texttt{PERSON} = 1 \text{ row/person}$$

while:

$$\texttt{CONDITION_OCCURRENCE} = 1 \text{ row/condition occurrence}$$

and:

$$\texttt{MEASUREMENT} = 1 \text{ row/measurement}$$

Joining these directly can produce many rows per patient.

Always ask:

> What does one row represent in this OMOP table?

**Standardization does not imply perfect data**

An OMOP database can be structurally valid while still containing problems with:

- missing data;
- incomplete vocabulary mappings;
- implausible values;
- duplicate events;
- inconsistent units;
- source-specific coding practices;
- incomplete capture of healthcare utilization.

Therefore:

$$\boxed{\text{Common schema} \neq \text{guaranteed data quality}}$$

**Practical tips**

- Start by identifying the clinical domain and corresponding OMOP table.
- Use standardized concept IDs for portable analyses.
- Preserve source concepts when investigating unexpected mappings.
- Join to `CONCEPT` when you need readable concept names.
- Use `CONCEPT_ANCESTOR` instead of manually enumerating large clinical concept families when appropriate.
- Check units before analyzing measurements.
- Determine table grain before joining multiple clinical domains.
- Use `DISTINCT` thoughtfully; it can hide an incorrect many-to-many join rather than actually fixing it.
- Validate cohort logic against clinical expectations, not just SQL syntax.

**Interview notes**

Be able to explain why OMOP exists: different healthcare organizations use different source systems and coding conventions, so OMOP maps those data into a common relational model and standardized vocabulary.

The most useful distinctions to remember are:

- OMOP versus FHIR;
- source concepts versus standard concepts;
- person-level versus event-level tables;
- `CONCEPT` versus `CONCEPT_ANCESTOR`;
- structural standardization versus actual data quality.

For analytical work, the combination of **SQL + OMOP vocabulary knowledge + careful cohort construction** is generally more important than memorizing every OMOP table.

# 3 Analysis and Modeling

## 3.1 Google Colab

**Google Colab** is a hosted Jupyter notebook environment useful for exploratory analysis, prototyping, teaching, and temporary GPU/TPU compute.

**Common shortcuts**

| Shortcut | Action |
|---|---|
| Shift + Enter | Run cell and move to next cell |
| Ctrl/Cmd + Enter | Run current cell |
| Alt/Option + Enter | Run cell and insert a new cell |
| Ctrl/Cmd + M, B | Insert cell below |
| Ctrl/Cmd + M, A | Insert cell above |
| Ctrl/Cmd + M, D | Delete current cell |

**Shell commands**

Prefix shell commands with `!`:

```
!pwd
!ls
!pip install package_name
!git clone https://github.com/user/project.git
```

The default working directory is usually:

```
/content
```

**Useful magic commands**

```
%pwd
%cd /content/project
%time expression
%timeit expression
%who
%whos
```

Time an entire cell:

```
%%time

# code
```

**Google Drive**

The runtime filesystem is temporary. Mount Drive when persistent storage is needed:

```
from google.colab import drive

drive.mount("/content/drive")
```

Files are commonly available under:

```
/content/drive/MyDrive/
```

### File upload/download

```
from google.colab import files

uploaded = files.upload()

files.download("results.csv")
```

### Packages

```
!pip install transformers
!pip install -q datasets accelerate
```

For reproducible work, pin important versions:

```
!pip install package_name==1.2.3
```

A restarted runtime may require packages to be installed again.

### GPU

Check the assigned GPU:

```
!nvidia-smi
```

Check availability with PyTorch:

```
import torch

torch.cuda.is_available()
torch.cuda.get_device_name(0)
```

Move a model to the available device:

```
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = model.to(device)
```

Selecting a GPU runtime does not automatically make all computations use the GPU.

**Good notebook structure**

```
1. Imports
2. Configuration and seeds
3. Data loading
4. Data processing
5. Modeling
6. Evaluation
7. Results / export
```

Before sharing a notebook, test:

```
Restart runtime -> Run all
```

This catches hidden dependencies caused by running cells out of order.

**Practical tips**

- Assume files in `/content` are temporary.
- Save important results and checkpoints to persistent storage.
- Keep installation and configuration cells near the top.
- Restart and run all cells before sharing a notebook.
- Avoid relying on variables created by out-of-order cell execution.
- Use `!nvidia-smi` to verify accelerator availability.
- Do not hard-code secrets into notebooks.
- Save checkpoints during long training jobs.

**Interview notes**

Colab is a hosted development environment, not a production deployment platform. Be comfortable discussing ephemeral runtimes, reproducibility, notebook state, package environments, and CPU/GPU execution.

## 3.2 SAS

**SAS** is a statistical programming and data-management environment widely used in healthcare, clinical research, government, finance, and regulated enterprise settings.

The two fundamental building blocks are:

- **DATA step**: create, transform, and filter data.
- **PROC step**: analyze, summarize, model, sort, or report data.

### DATA step

```
DATA adults;
    SET patients;
    WHERE age >= 18;
RUN;
```

Create variables:

```
DATA patients2;
    SET patients;

    bmi = weight_kg / (height_m ** 2);

    IF bmi >= 30 THEN obese = 1;
    ELSE obese = 0;
RUN;
```

### Libraries

Assign a permanent library:

```
LIBNAME study "/path/to/data";
```

Reference a dataset:

```
study.patients
```

Temporary datasets are commonly stored in:

```
WORK
```

The `WORK` library disappears when the SAS session ends.

### Inspect data

```
PROC CONTENTS DATA=study.patients;
RUN;

PROC PRINT DATA=study.patients (OBS=10);
RUN;
```

### KEEP and DROP

```
DATA analysis;
    SET patients;
    KEEP patient_id age treatment outcome;
RUN;
```

or:

```
DATA analysis;
    SET patients;
    DROP address phone_number;
RUN;
```

### WHERE versus IF

```
WHERE age >= 18;
```

filters observations as they are read and generally uses variables already present in the input data.

```
IF age >= 18;
```

operates during DATA-step processing and can use variables created within the step.

### Sorting

```
PROC SORT DATA=patients
          OUT=patients_sorted;
    BY patient_id visit_date;
RUN;
```

### BY-group processing

After sorting:

```
DATA first_visit;
    SET patients_sorted;

    BY patient_id;

    IF FIRST.patient_id;
RUN;
```

Useful temporary indicators:

```
FIRST.patient_id
LAST.patient_id
```

Keep the latest record:

```
PROC SORT DATA=patients;
    BY patient_id visit_date;
RUN;

DATA latest;
    SET patients;
    BY patient_id;
```

```
    IF LAST.patient_id;
RUN;
```

### Missing values

Numeric missing value:

```
.
```

Check missingness:

```
IF MISSING(age) THEN ...
```

Character missing values are represented by blanks.

### PROC FREQ

```
PROC FREQ DATA=patients;
    TABLES treatment;
RUN;
```

Cross-tabulation:

```
PROC FREQ DATA=patients;
    TABLES treatment*outcome / CHISQ;
RUN;
```

### PROC MEANS

```
PROC MEANS DATA=patients
           N MEAN STD MEDIAN MIN MAX;
    VAR age bmi;
RUN;
```

Grouped summary:

```
PROC MEANS DATA=patients;
    CLASS treatment;
    VAR outcome;
RUN;
```

### PROC SQL

```
PROC SQL;
    SELECT
        treatment,
        COUNT(*) AS n,
        AVG(outcome) AS mean_outcome
```

```
    FROM patients
    GROUP BY treatment;
QUIT;
```

Create a dataset:

```
PROC SQL;
    CREATE TABLE adults AS
    SELECT *
    FROM patients
    WHERE age >= 18;
QUIT;
```

### Linear regression

```
PROC REG DATA=patients;
    MODEL outcome = age bmi treatment;
RUN;
QUIT;
```

### Logistic regression

```
PROC LOGISTIC DATA=patients;
    CLASS treatment / PARAM=REF;

    MODEL outcome(EVENT='1') =
        age bmi treatment;
RUN;
```

### Formats

Formats change how values are displayed without changing the underlying stored values.

```
PROC FORMAT;
    VALUE treatment_fmt
        0 = "Control"
        1 = "Treatment";
RUN;

DATA patients;
    SET patients;
    FORMAT treatment treatment_fmt.;
RUN;
```

### Macro variables

```
%LET year = 2026;

PROC PRINT DATA=patients;
```

```
    WHERE year = &year;
RUN;
```

Simple macro:

```
%MACRO summarize(var);

PROC MEANS DATA=patients;
    VAR &var;
RUN;

%MEND;

%summarize(age)
```

Macros perform text/code generation and are different from DATA-step variables.

**Practical tips**

- Know whether data are temporary (`WORK`) or permanent.
- Use `PROC CONTENTS` when opening an unfamiliar dataset.
- Sort before relying on BY-group processing.
- Know `FIRST.` and `LAST.` for grouped records.
- Distinguish displayed formats from stored values.
- Check the SAS log for warnings and errors after execution.
- Handle missing values explicitly.

**Interview notes**

Know DATA versus PROC steps, `SET`, `WHERE` versus `IF`, libraries, BY-group processing, `FIRST.`/`LAST.`, formats, missing values, `PROC SQL`, `PROC FREQ`, and `PROC MEANS`.

### 3.3 Stata

**Stata** is a statistical programming environment widely used in economics, econometrics, epidemiology, public policy, and social science.

General command structure:

```
command varlist [if] [in], options
```

Example:

```
summarize income if age >= 18, detail
```

### Load and save

```
use "data.dta", clear

save "clean_data.dta", replace

import delimited "data.csv", clear

export delimited using "results.csv", replace
```

### Inspect data

```
describe
codebook
summarize
summarize income, detail
list in 1/10
browse
count
tabulate treatment
```

Cross-tabulation:

```
tabulate treatment outcome
```

### Generate and replace

```
generate bmi = weight_kg / (height_m^2)

generate adult = age >= 18

replace income = . if income < 0
```

`gen` is a common abbreviation for `generate`.

### Missing values

Standard numeric missing:

```
.
```

Extended missing values:

```
.a .b .c ... .z
```

Important Stata behavior: numeric missing values compare greater than ordinary numeric values.

Therefore:

```
keep if age > 18
```

may retain missing ages.

Safer:

```
keep if age > 18 & !missing(age)
```

### Keep and drop

Variables:

```
keep patient_id age treatment outcome

drop address phone_number
```

Observations:

```
keep if age >= 18

drop if missing(outcome)
```

### Sort and bysort

```
sort patient_id date

bysort patient_id (date): gen visit_number = _n
```

Within a group:

- `_n`: current observation number.
- `_N`: total observations in the group.

Keep first record per patient:

```
bysort patient_id (date): keep if _n == 1
```

Count observations per patient:

```
bysort patient_id: gen n_visits = _N
```

### egen

Grouped mean:

```
bysort state: egen mean_income = mean(income)
```

Group identifier:

```
egen firm_id = group(firm_name)
```

Row total:

```
egen total_score = rowtotal(score1 score2 score3)
```

### Collapse

```
collapse (mean) income age ///
         (count) n=id, by(state)
```

`collapse` replaces the current dataset in memory.

Use:

```
preserve

collapse (mean) income, by(state)

restore
```

when the original data are still needed.

### Reshape

Wide to long:

```
reshape long income, i(id) j(year)
```

Long to wide:

```
reshape wide income, i(id) j(year)
```

### Merge

```
merge 1:1 patient_id using "demographics.dta"

merge 1:m id using "visits.dta"

merge m:1 firm_id using "firm_data.dta"
```

Always inspect:

```
tab _merge
```

Typical values:

| _merge | Meaning |
|---|---|
| 1 | Master only |
| 2 | Using only |
| 3 | Matched |

Avoid `merge m:m` for ordinary relational data. It usually indicates that the intended key structure has not been resolved correctly.

### Linear regression

```
regress y x1 x2 x3

regress y x1 x2 x3, vce(robust)

regress y x1 x2 x3, vce(cluster firm_id)
```

### Logistic regression

```
logit y x1 x2 x3

logistic y x1 x2 x3
```

### Factor variables

```
regress y x1 i.industry

regress y c.age i.treatment

regress y c.age##i.treatment
```

Useful distinction:

- `#`: interaction term.
- `##`: main effects plus interaction.

### Panel data

```
xtset firm_id year

xtreg y x1 x2, fe

xtreg y x1 x2, re
```

### Time series

```
tsset date

generate lag_y = L.y

generate diff_y = D.y

generate lead_y = F.y
```

### Local macros

```
local controls age income education

regress y treatment `controls'
```

Prefer local macros over global macros when possible.

**Do-files**

A reproducible Stata workflow should usually live in a `.do` file:

```
clear all
set more off

cd "/path/to/project"

log using "analysis.log", replace

* Load data
* Clean data
* Analyze
* Export results

log close
```

**Practical tips**

- Work from do-files rather than relying on command history.
- Inspect `_merge` after every merge.
- Avoid `merge m:m` unless there is a specific reason.
- Remember Stata's unusual numeric missing-value ordering.
- Use `preserve`/`restore` around destructive temporary operations.
- Use factor-variable notation rather than manually creating every dummy.
- Declare panel or time-series structure before using corresponding operators.

**Interview notes**

Know `generate`, `replace`, `egen`, `bysort`, `_n`, `_N`, `merge`, `reshape`, and `collapse`.

For econometrics roles, also know factor variables, robust versus clustered standard errors, panel declarations, fixed versus random effects, and basic lag/difference operators.

## 3.4 JAX

**JAX** is a numerical computing library with a NumPy-like interface, automatic differentiation, JIT compilation, vectorization, and accelerator support.

The three transformations to remember are:

- `grad`: automatic differentiation.
- `jit`: just-in-time compilation.
- `vmap`: automatic vectorization.

**Arrays**

```
import jax
import jax.numpy as jnp

x = jnp.array([1.0, 2.0, 3.0])

jnp.mean(x)
jnp.sum(x)
jnp.exp(x)
jnp.dot(x, x)
```

JAX intentionally resembles NumPy:

```
import numpy as np
import jax.numpy as jnp
```

but the execution models differ.

**Automatic differentiation**

```
from jax import grad

def f(x):
    return x**2 + 3*x

df = grad(f)

df(2.0)
```

For a loss:

```
def loss(w, x, y):
    pred = x @ w
    return jnp.mean((pred - y)**2)

grad_loss = grad(loss)

grads = grad_loss(w, x, y)
```

By default, `grad()` differentiates with respect to the first argument.

Specify another argument with:

```
grad(loss, argnums=0)
```

### Value and gradient together

```
from jax import value_and_grad

loss_and_grad = value_and_grad(loss)

value, grads = loss_and_grad(w, x, y)
```

### JIT compilation

```
from jax import jit

@jit
def f(x):
    return jnp.sin(x) + jnp.cos(x)
```

or:

```
compiled_f = jit(f)
```

The first compatible call includes tracing and compilation overhead. Later calls can be much faster.

Do not benchmark only the first JIT call when measuring steady-state performance.

### Vectorization

Suppose a function handles one observation:

```
def predict(w, x):
    return x @ w
```

Vectorize it:

```
from jax import vmap

batch_predict = vmap(
    predict,
    in_axes=(None, 0)
)

predictions = batch_predict(w, X)
```

Here:

- `None`: reuse the same `w`.
- `0`: vectorize across axis 0 of `X`.

### Immutable arrays

Do not use NumPy-style mutation:

```
# Avoid
x[0] = 10
```

Use functional updates:

```
x = x.at[0].set(10)

x = x.at[0].add(5)

x = x.at[idx].set(values)
```

### Randomness

JAX uses explicit pseudo-random keys:

```
from jax import random

key = random.key(42)

x = random.normal(
    key,
    shape=(100,)
)
```

Split keys when independent randomness is required:

```
key, subkey = random.split(key)

x = random.normal(
    subkey,
    shape=(100,)
)
```

Do not repeatedly reuse the same key when independent draws are intended.

### PyTrees

Model parameters can be stored in nested structures:

```
params = {
    "layer1": {
        "weights": jnp.ones((10, 5)),
        "bias": jnp.zeros(5)
    },
    "layer2": {
        "weights": jnp.ones((5, 1)),
```

```
        "bias": jnp.zeros(1)
    }
}
```

JAX transformations understand many nested structures such as dictionaries, lists, and tuples as PyTrees.

**Devices**

```
jax.devices()

jax.default_backend()
```

JAX can execute on CPU, GPU, or TPU depending on the environment.

**JAX versus NumPy**

| Feature | NumPy | JAX |
|---|---|---|
| Array interface | Standard | NumPy-like |
| Autodiff | No | Yes |
| JIT compilation | No | Yes |
| GPU/TPU | Not native | Supported |
| Mutation | Supported | Functional style |
| Randomness | Stateful-style | Explicit keys |

**Practical tips**

- Write pure, array-oriented functions when possible.
- Do not mutate JAX arrays in place.
- Split PRNG keys deliberately.
- Expect compilation overhead on the first JIT call.
- Avoid unnecessary shape changes that trigger recompilation.
- Use `vmap` instead of Python loops when appropriate.
- Check which device JAX is actually using.

**Interview notes**

The key idea is that JAX is built around transformations of functions. Know `grad`, `jit`, and `vmap`, plus immutability, explicit PRNG keys, tracing, and PyTrees.

## 3.5 Hugging Face

**Hugging Face** is an ecosystem for pretrained models, tokenizers, datasets, and model sharing. Important libraries include `transformers`, `datasets`, and the Hugging Face Hub.

### Pipeline

For quick inference:

```
from transformers import pipeline

classifier = pipeline(
    "text-classification"
)

result = classifier(
    "The model performed very well."
)
```

Specify a model:

```
classifier = pipeline(
    "text-classification",
    model="model-name"
)
```

Common tasks include:

```
text-classification
token-classification
question-answering
summarization
translation
text-generation
image-classification
automatic-speech-recognition
```

### Tokenizer and model

For more control:

```
from transformers import AutoTokenizer
from transformers import AutoModel

tokenizer = AutoTokenizer.from_pretrained(
    "model-name"
)

model = AutoModel.from_pretrained(
    "model-name"
)
```

Common task-specific classes:

```
AutoModelForSequenceClassification
AutoModelForTokenClassification
AutoModelForCausalLM
AutoModelForSeq2SeqLM
AutoModelForQuestionAnswering
```

Use a task-specific model class when the task requires a corresponding output head.

**Tokenization**

Single input:

```
inputs = tokenizer(
    "Example sentence.",
    return_tensors="pt"
)
```

Batch:

```
inputs = tokenizer(
    texts,
    padding=True,
    truncation=True,
    return_tensors="pt"
)
```

Important options:

- `padding`: pad sequences to compatible lengths.
- `truncation`: truncate overly long inputs.
- `max_length`: set an explicit sequence-length limit.
- `return_tensors`: choose the tensor framework.

**Tokens versus words**

Tokens are not necessarily words. A tokenizer may divide a word into several subword tokens.

This matters for:

- context-window limits;
- padding;
- truncation;
- memory use;
- generation length.

### Attention masks

Padded batches commonly contain:

```
input_ids
attention_mask
```

Conceptually:

```
Tokens: [A, B, C, PAD, PAD]
Mask:   [1, 1, 1, 0,   0]
```

The attention mask identifies real tokens versus padding positions.

### Text generation

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained(
    "model-name"
)

inputs = tokenizer(
    "Explain Bayesian inference:",
    return_tensors="pt"
)

outputs = model.generate(
    **inputs,
    max_new_tokens=100
)

text = tokenizer.decode(
    outputs[0],
    skip_special_tokens=True
)
```

Common generation parameters:

```
max_new_tokens
max_length
temperature
top_p
top_k
do_sample
num_beams
repetition_penalty
```

Useful distinction:

- `max_length`: limits the total sequence length, including the input and generated tokens.
- `max_new_tokens`: limits only the number of newly generated tokens.

For controlling output length, `max_new_tokens` is usually easier to reason about.

**Greedy generation versus sampling**

Deterministic generation:

```
outputs = model.generate(
    **inputs,
    do_sample=False,
    max_new_tokens=100
)
```

Sampling:

```
outputs = model.generate(
    **inputs,
    do_sample=True,
    temperature=0.8,
    top_p=0.9,
    max_new_tokens=100
)
```

General interpretation:

- Lower temperature makes sampling more concentrated.
- Higher temperature increases randomness.
- `top_p` restricts sampling to a probability-mass subset.
- `top_k` restricts sampling to the top $k$ candidate tokens.

**Datasets**

```
from datasets import load_dataset

dataset = load_dataset("dataset-name")
```

Common splits:

```
dataset["train"]
dataset["validation"]
dataset["test"]
```

Transform a dataset:

```
def tokenize(batch):
    return tokenizer(
```

```
        batch["text"],
        truncation=True
    )

tokenized = dataset.map(
    tokenize,
    batched=True
)
```

Filter:

```
filtered = dataset.filter(
    lambda x: x["label"] == 1
)
```

### Model cards

Before using a checkpoint, inspect its model card for:

- architecture;
- intended use;
- training data;
- evaluation results;
- limitations;
- license;
- preprocessing requirements;
- supported context length.

### Base versus fine-tuned models

- **Base model**: pretrained general-purpose model.
- **Fine-tuned model**: model adapted to a task or desired behavior.
- **Instruction model**: model fine-tuned to follow natural-language instructions.

Use a checkpoint appropriate for the intended task.

### Full fine-tuning versus LoRA

- **Full fine-tuning**: updates most or all model parameters.
- **LoRA**: learns smaller low-rank parameter updates while leaving most original model parameters frozen.

LoRA and other parameter-efficient methods can substantially reduce memory and storage requirements.

**Practical tips**

- Match the model class to the task.
- Use the tokenizer associated with the checkpoint.
- Check context limits before processing long documents.
- Set padding and truncation deliberately.
- Prefer `max_new_tokens` when controlling generated output length.
- Inspect model cards and licenses before using a checkpoint.
- Batch inference when practical.
- Evaluate models on the actual target task rather than relying only on published benchmark scores.

**Interview notes**

Know the distinction between:

- model and tokenizer;
- base and fine-tuned models;
- tokens and words;
- padding and truncation;
- token IDs and attention masks;
- full fine-tuning and LoRA;
- greedy generation and sampling.

Also understand why the high-level `pipeline` interface is convenient for quick inference while loading the tokenizer and model separately gives more control.

## 3.6 LangChain

**LangChain** is an orchestration framework for applications built around language models. It provides abstractions for prompts, models, structured output, documents, retrieval, tools, and agentic workflows.

LangChain is not itself a language model.

**Core components**

| Component | Purpose |
|---|---|
| Chat Model | Language-model interface |
| Prompt Template | Constructs model input |
| Output Parser | Converts model output into another representation |
| Document | Text plus associated metadata |
| Retriever | Finds relevant documents |
| Tool | Callable capability exposed to a workflow |
| Agent | Model-directed tool-use workflow |

### Prompt templates

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    (
        "system",
        "You are a statistical assistant."
    ),
    (
        "human",
        "Explain {topic} in {n} sentences."
    )
])
```

Fill variables:

```
prompt.invoke({
    "topic": "bootstrap sampling",
    "n": 3
})
```

### Runnable composition

Components can be composed with the pipe operator:

```
chain = prompt | model | parser
```

Invoke:

```
result = chain.invoke({
    "topic": "bootstrap sampling",
    "n": 3
})
```

Batch:

```
results = chain.batch([
    {
```

```
        "topic": "bootstrap",
        "n": 2
    },
    {
        "topic": "cross-validation",
        "n": 2
    }
])
```

Stream:

```
for chunk in chain.stream(inputs):
    print(chunk, end="")
```

### Structured output

Application code often needs predictable fields rather than arbitrary prose.

Conceptually:

```
structured_model = model.with_structured_output(
    OutputSchema
)

result = structured_model.invoke(input)
```

Structured output is useful when downstream code expects values such as:

```
classification
confidence
reason
```

Prefer structured output over manually extracting values from free-form text when possible.

### Documents

```
from langchain_core.documents import Document

doc = Document(
    page_content=(
        "Clinical trial eligibility..."
    ),
    metadata={
        "source": "trial_123",
        "section": "eligibility"
    }
)
```

Useful metadata includes:

- source;
- document ID;
- page or section;
- date;
- category;
- other filtering/provenance information.

### RAG workflow

A typical retrieval-augmented generation workflow has two stages.

Indexing:

```
Documents
-> Chunking
-> Embeddings
-> Vector Store
```

Querying:

```
Question
-> Retriever
-> Relevant Chunks
-> Model
-> Answer
```

Retrieve documents:

```
docs = retriever.invoke(query)
```

### Chunking

Important chunking decisions include:

- chunk size;
- overlap;
- document structure;
- semantic boundaries;
- metadata preservation.

Large chunks preserve more context but may introduce irrelevant information. Very small chunks can improve retrieval precision but may lose necessary surrounding context.

There is no universally optimal chunk size.

### Debugging RAG

When a RAG answer is wrong, separate the problem into:

1. Was the correct evidence retrieved?
2. If yes, did the model use that evidence correctly?

If retrieval is wrong, inspect:

- document parsing;
- chunking;
- embeddings;
- metadata filters;
- retrieval depth;
- query formulation.

Do not immediately change the generation prompt when the relevant information was never retrieved.

### Tools

A tool exposes a controlled capability:

```
from langchain_core.tools import tool

@tool
def calculate_score(
    x: float,
    y: float
) -> float:
    """Calculate the requested score."""
    return x + y
```

Good tools should have:

- a clear name;
- a narrow purpose;
- well-defined inputs;
- predictable outputs;
- appropriate permissions.

### Agents

Agents allow a model to decide which action or tool to use next.

Use an agent when the sequence of actions genuinely needs to be dynamic.

For a known workflow such as:

```
Input -> Retrieve -> Generate -> Validate
```

a deterministic pipeline is usually simpler and easier to test.

**State**

LLM applications must explicitly manage state such as:

- prior messages;
- conversation summaries;
- user/session information;
- retrieved context;
- workflow state.

Do not assume that the model automatically maintains persistent application state.

**Prompt injection**

Retrieved documents and user-provided content should be treated as untrusted input.

A retrieved document could contain text such as:

```
Ignore all previous instructions and...
```

Instructions contained inside retrieved content should not automatically be treated as trusted application instructions.

This is especially important when a model can invoke tools or perform actions.

**Practical tips**

- Use deterministic pipelines when the workflow is known.
- Use agents only when dynamic action selection is actually needed.
- Prefer structured outputs for downstream application logic.
- Preserve document metadata throughout RAG workflows.
- Inspect retrieved chunks directly when debugging RAG.
- Evaluate retrieval separately from generation.
- Keep tools narrow and clearly described.
- Restrict tool permissions to what the workflow needs.
- Treat retrieved text as untrusted content.

- Manage conversation and workflow state explicitly.

**Interview notes**

LangChain is an orchestration framework, not an LLM.

Know the distinction between:

- prompt, model, and parser;
- document and retriever;
- embeddings and retrieval;
- retrieval and generation;
- deterministic pipelines and agents;
- free-form and structured output;
- model context and persistent application state.

For RAG interviews, a strong debugging approach is to inspect retrieval first. If the correct evidence was not retrieved, modifying the generation prompt does not address the underlying retrieval failure.

# 4 Validation

## 4.1 MLflow

**MLflow** is an open-source platform for tracking machine-learning experiments and managing models throughout the ML lifecycle.

For day-to-day modeling, its most useful purpose is answering questions such as:

- Which model produced this result?
- Which hyperparameters were used?
- Which run performed best?
- Where are the plots and model artifacts?
- Which model version should be used downstream?

The core hierarchy is:

```
Experiment
    -> Run
        -> Parameters
        -> Metrics
        -> Tags
        -> Artifacts
        -> Models
```

### Core terminology

| Term | Meaning |
|---|---|
| Experiment | Collection of related runs |
| Run | One execution of an experiment |
| Parameter | Input/configuration value for a run |
| Metric | Numeric performance value |
| Artifact | File produced by a run |
| Tag | Searchable metadata attached to a run |
| Tracking Server | Stores and serves experiment metadata |
| Artifact Store | Stores files such as models and plots |
| Model Registry | Organizes registered models and versions |

### Install MLflow

```
pip install mlflow
```

Check installation:

```
mlflow --version
```

### Start the MLflow UI/server

For local work:

```
mlflow server --port 5000
```

Then open:

```
http://localhost:5000
```

Set the tracking location in Python:

```
import mlflow

mlflow.set_tracking_uri(
    "http://localhost:5000"
)
```

For simple local experimentation, MLflow can also be used without operating a shared remote tracking server.

### Experiments

Create or select an experiment:

```
import mlflow

mlflow.set_experiment(
```

```
    "classification-models"
)
```

Use experiments to separate meaningful groups of work, such as:

```
baseline-models
feature-engineering
hyperparameter-search
forecasting-experiments
production-candidates
```

Avoid creating a completely new experiment for every individual run.

**Runs**

A run represents one execution of a model or experiment.

```
with mlflow.start_run():

    # train model
    # evaluate model
    # log results
    pass
```

A typical pattern is:

```
with mlflow.start_run(
    run_name="random-forest-baseline"
):
    # experiment code
    pass
```

Give important runs meaningful names rather than relying only on generated identifiers.

**Parameters**

Parameters describe the configuration used for a run.

```
mlflow.log_param(
    "learning_rate",
    0.001
)

mlflow.log_param(
    "batch_size",
    64
)
```

Log several parameters:

```
mlflow.log_params({
    "learning_rate": 0.001,
    "batch_size": 64,
    "hidden_size": 128,
    "optimizer": "adam"
})
```

Typical parameters include:

- learning rate;
- batch size;
- number of trees;
- regularization strength;
- model architecture;
- random seed;
- preprocessing choices.

**Metrics**

Metrics are numeric measurements of model performance.

```
mlflow.log_metric(
    "accuracy",
    accuracy
)

mlflow.log_metric(
    "f1",
    f1
)
```

Log several metrics:

```
mlflow.log_metrics({
    "accuracy": accuracy,
    "precision": precision,
    "recall": recall,
    "f1": f1
})
```

For iterative training, log the same metric over multiple steps:

```
for epoch in range(num_epochs):
```

```
    train_loss = train_one_epoch(...)

    mlflow.log_metric(
        "train_loss",
        train_loss,
        step=epoch
    )
```

This produces a metric history rather than only a final value.

**Parameters versus metrics**

Keep the distinction clear:

- **Parameter**: configuration supplied to the experiment.
- **Metric**: measured result produced by the experiment.

For example:

```
learning_rate = 0.001    -> parameter
batch_size    = 64       -> parameter

validation_loss = 0.32   -> metric
accuracy        = 0.91   -> metric
```

**Tags**

Tags store descriptive metadata:

```
mlflow.set_tag(
    "model_type",
    "random_forest"
)

mlflow.set_tag(
    "dataset",
    "customer_churn_v2"
)
```

Useful tags may describe:

- model family;
- dataset version;
- experiment purpose;
- development stage;
- feature set;

- notes about a run.

**Artifacts**

Artifacts are files associated with a run.

Examples include:

- plots;
- confusion matrices;
- calibration plots;
- feature-importance figures;
- prediction files;
- configuration files;
- model files.

Log one artifact:

```
mlflow.log_artifact(
    "confusion_matrix.png"
)
```

Organize it inside the run:

```
mlflow.log_artifact(
    "confusion_matrix.png",
    artifact_path="plots"
)
```

Log an entire directory:

```
mlflow.log_artifacts(
    "results/",
    artifact_path="results"
)
```

**Complete tracking pattern**

A typical experiment might look like:

```
import mlflow

mlflow.set_experiment(
    "classification-models"
)

with mlflow.start_run(
```

```
    run_name="rf-baseline"
):

    mlflow.log_params({
        "n_estimators": 500,
        "max_depth": 10,
        "random_state": 42
    })

    model.fit(X_train, y_train)

    predictions = model.predict(X_valid)

    accuracy = accuracy_score(
        y_valid,
        predictions
    )

    f1 = f1_score(
        y_valid,
        predictions
    )

    mlflow.log_metrics({
        "accuracy": accuracy,
        "f1": f1
    })

    mlflow.log_artifact(
        "confusion_matrix.png"
    )
```

The important idea is that the model result and the configuration that produced it are stored together.

### Autologging

MLflow can automatically capture common information for supported ML frameworks.

For example:

```
import mlflow
import mlflow.sklearn

mlflow.sklearn.autolog()

with mlflow.start_run():
    model.fit(
```

```
        X_train,
        y_train
    )
```

Autologging can capture information such as:

- model parameters;
- selected metrics;
- fitted models;
- framework metadata.

Autologging is convenient, but manually log project-specific information that the automatic integration does not know about.

### Logging models

For a scikit-learn model:

```
import mlflow.sklearn

mlflow.sklearn.log_model(
    sk_model=model,
    name="model"
)
```

Logging the model with MLflow keeps the fitted model associated with the run rather than saving an unexplained model file somewhere on disk.

### Loading a logged model

A model can later be loaded from its MLflow model location.

For a scikit-learn model:

```
import mlflow.sklearn

model = mlflow.sklearn.load_model(
    model_uri
)
```

This is useful when evaluation, deployment, or batch prediction should use the exact model produced by a tracked run.

### Model Registry

The Model Registry provides a higher-level organization for models intended to move beyond individual experiment runs.

Useful concepts include:

- registered model;
- model version;
- model alias;
- model metadata.

A common workflow is:

```
Train candidate
-> Track run
-> Compare validation results
-> Register selected model
-> Assign useful alias
-> Use registered model downstream
```

Aliases can represent roles such as:

```
champion
candidate
production
```

The exact naming convention is a team decision; the important point is to avoid identifying production models only by fragile local filenames.

### Comparing runs

One of MLflow's most useful features is comparing experiments across parameters and metrics.

Programmatically:

```
runs = mlflow.search_runs(
    experiment_names=[
        "classification-models"
    ]
)
```

The result can be inspected like a table:

```
runs[[
    "run_id",
    "params.max_depth",
    "metrics.accuracy",
    "metrics.f1"
]]
```

This makes it easier to answer questions such as:

- Which run had the highest validation F1?

- Which hyperparameters produced the best result?
- Did increasing model complexity actually improve performance?

**What to log**

A useful minimum for most experiments is:

| Category | Examples |
|---|---|
| Parameters | Hyperparameters, seed, preprocessing settings |
| Metrics | Validation loss, RMSE, F1, AUROC, calibration metrics |
| Tags | Model family, dataset version, experiment purpose |
| Artifacts | Plots, tables, predictions, configuration files |
| Model | Fitted model used to generate the results |

For probabilistic models, useful artifacts may include:

- calibration curves;
- reliability diagrams;
- residual plots;
- PIT histograms;
- prediction intervals;
- evaluation tables.

**Validation versus test data**

MLflow tracks results, but it does not prevent bad experimental design.

Keep the roles of datasets distinct:

- **Training set**: fit model parameters.
- **Validation set**: tune models and compare candidates.
- **Test set**: estimate final generalization performance.

If the test set is repeatedly used to choose models, it effectively becomes another validation set.

MLflow makes comparisons easier, so it is especially important not to use that convenience to repeatedly optimize against the final test set.

**Metric selection**

Do not select a model solely because one metric is numerically highest.

Examples:

- Accuracy can be misleading under class imbalance.

- AUROC and AUPRC emphasize different aspects of classification.
- RMSE penalizes large errors more heavily than MAE.
- Good likelihood does not guarantee good calibration.
- Average performance can hide poor subgroup performance.

Log the metrics that correspond to the actual modeling objective.

**Data leakage**

Experiment tracking cannot fix leakage.

Common examples include:

- preprocessing the full dataset before splitting;
- using future information in time-series features;
- selecting features using the test set;
- allowing the same patient or entity to appear across inappropriate splits;
- fitting normalization parameters using validation/test observations.

A perfectly tracked leaked experiment is still invalid.

**Reproducibility**

Tracking hyperparameters alone is not enough for complete reproducibility.

Also consider recording:

- random seed;
- dataset/version;
- train/validation/test split;
- preprocessing configuration;
- package/environment information;
- code version or commit;
- model architecture;
- important feature definitions.

The goal is to be able to answer:

> What exactly produced this model and this result?

**Run naming**

Poor:

```
run1
run2
test
test2
final
final_final
```

Better:

```
rf-baseline
xgb-class-weighted
lstm-window-30
transformer-small
```

Use parameters and tags for detailed configuration rather than encoding every setting into an enormous run name.

**Practical tips**

- Create experiments around meaningful projects or experiment families.
- Give important runs descriptive names.
- Log configuration before or during training rather than reconstructing it afterward.
- Log validation metrics, not just training metrics.
- Save important plots as artifacts.
- Record random seeds and dataset versions.
- Use tags for information that helps filter and organize runs.
- Use autologging for convenience, then add project-specific metrics and artifacts manually.
- Compare runs systematically rather than relying on notebook output or memory.
- Do not repeatedly tune against the final test set.
- Keep model selection criteria consistent across runs.
- Track enough information to reproduce the winning run.

**Common mistakes**

- Logging only the final model and not the configuration that produced it.
- Logging only training performance.
- Changing datasets without recording which version was used.
- Comparing runs evaluated on different splits as if they were equivalent.

- Using inconsistent metric definitions across experiments.
- Tracking hundreds of runs without useful names, tags, or organization.
- Assuming experiment tracking automatically guarantees reproducibility.
- Choosing the best model by repeatedly inspecting test-set performance.

**Interview notes**

The main purpose of MLflow is **experiment tracking and model lifecycle management**. It helps connect model configurations, metrics, artifacts, and models so experiments can be compared and reproduced.

Be comfortable explaining:

- experiment versus run;
- parameter versus metric;
- metric versus artifact;
- manual logging versus autologging;
- model logging versus model registration;
- why dataset and code versions matter for reproducibility;
- why experiment tracking does not prevent leakage or poor validation design.

A useful interview workflow is:

```
Define experiment
-> Start run
-> Log configuration
-> Train model
-> Evaluate on validation data
-> Log metrics and artifacts
-> Compare runs
-> Select candidate
-> Final evaluation
-> Register/version selected model
```

If asked why MLflow is useful instead of simply saving models as `model.pkl`, emphasize that a model file alone does not tell you which data, parameters, code, metrics, or experimental conditions produced it.

# 5 Deployment

## 5.1 Docker

**Docker** packages an application and its dependencies into a **container** so that it can run consistently across development, testing, and deployment environments.

The core idea is:

```
Application Code
+ Runtime
+ Dependencies
+ Configuration
= Container Image
```

A running instance of that image is a **container**.

### Core terminology

| Term | Meaning |
|---|---|
| Dockerfile | Instructions for building an image |
| Image | Immutable template used to create containers |
| Container | Running instance of an image |
| Registry | Repository for storing and distributing images |
| Volume | Persistent storage managed outside the container filesystem |
| Bind Mount | Host directory mounted directly into a container |
| Port Mapping | Maps a host port to a container port |
| Docker Compose | Defines and runs multi-container applications |

The distinction to remember is:

```
Dockerfile -> Image -> Container
```

### Check Docker

```
docker --version
docker info
```

### Images

List local images:

```
docker images
```

Pull an image:

```
docker pull python:3.12-slim
```

Remove an image:

```
docker rmi image_name
```

### Containers

List running containers:

```
docker ps
```

List all containers:

```
docker ps -a
```

Start:

```
docker start container_name
```

Stop:

```
docker stop container_name
```

Restart:

```
docker restart container_name
```

Remove:

```
docker rm container_name
```

Force-remove a running container:

```
docker rm -f container_name
```

### Dockerfile

A simple Python application:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir \
    -r requirements.txt

COPY . .

CMD ["python", "app.py"]
```

Important instructions:

| Instruction | Purpose |
|---|---|
| FROM | Select base image |
| WORKDIR | Set working directory |
| COPY | Copy files into image |
| RUN | Execute command while building image |
| ENV | Define environment variable |
| EXPOSE | Document intended container port |
| CMD | Default command when container starts |
| ENTRYPOINT | Define the executable for the container |

### FROM

Every Dockerfile normally begins with a base image:

```
FROM python:3.12-slim
```

Smaller base images generally reduce image size and unnecessary packages.

### WORKDIR

```
WORKDIR /app
```

Subsequent commands operate relative to this directory.

### COPY

```
COPY requirements.txt .
```

Copy the project:

```
COPY . .
```

### RUN

`RUN` executes while the image is being built:

```
RUN pip install --no-cache-dir \
    -r requirements.txt
```

The resulting filesystem changes become part of the image.

### CMD

`CMD` specifies the default command when the container starts:

```
CMD ["python", "app.py"]
```

Prefer the JSON/exec form shown above rather than:

```
CMD python app.py
```

### Build an image

From the directory containing the Dockerfile:

```
docker build -t my-app .
```

The final period means:

```
use the current directory as the build context
```

Tag a version:

```
docker build -t my-app:1.0 .
```

List images:

```
docker images
```

### Run a container

```
docker run my-app
```

Run with a name:

```
docker run --name my-container my-app
```

Run in the background:

```
docker run -d \
    --name my-container \
    my-app
```

Here:

```
-d
```

means detached/background mode.

Automatically remove the container when it exits:

```
docker run --rm my-app
```

This is useful for temporary jobs and testing.

### Port mapping

Suppose the application listens on port 8000 inside the container.

Run:

```
docker run -p 8000:8000 my-app
```

The syntax is:

```
-p HOST_PORT:CONTAINER_PORT
```

For example:

```
docker run -p 8080:8000 my-app
```

means:

```
localhost:8080
    -> container:8000
```

The host and container ports do not need to match.

### EXPOSE

A Dockerfile may contain:

```
EXPOSE 8000
```

This documents that the application expects to listen on port 8000.

It does **not** automatically publish the port to the host.

You still need:

```
docker run -p 8000:8000 my-app
```

#### Environment variables

Pass configuration when starting the container:

```
docker run \
    -e MODEL_NAME=my-model \
    -e LOG_LEVEL=INFO \
    my-app
```

Inside Python:

```
import os

model_name = os.getenv(
    "MODEL_NAME"
)
```

Use an environment file:

```
docker run --env-file .env my-app
```

Example `.env`:

```
MODEL_NAME=my-model
LOG_LEVEL=INFO
DATABASE_HOST=db
```

Do not bake secrets directly into the image or commit them into source control.

**Logs**

View container logs:

```
docker logs my-container
```

Follow logs continuously:

```
docker logs -f my-container
```

Show recent logs:

```
docker logs --tail 100 my-container
```

**Execute a command inside a running container**

Open a shell:

```
docker exec -it my-container sh
```

If Bash is installed:

```
docker exec -it my-container bash
```

Run one command:

```
docker exec my-container ls /app
```

**Inspect a container**

```
docker inspect my-container
```

Useful for checking:

- environment variables;
- networking;
- mounts;

- container configuration;
- image information.

**View resource usage**

```
docker stats
```

This is useful when debugging unexpectedly high CPU or memory usage.

**Container storage**

A container's writable filesystem should generally be treated as temporary.

Removing the container can remove data stored only inside that container.

Use external storage for persistent data.

**Volumes**

Create a volume:

```
docker volume create app-data
```

Mount it:

```
docker run \
    -v app-data:/app/data \
    my-app
```

List volumes:

```
docker volume ls
```

Inspect:

```
docker volume inspect app-data
```

**Bind mounts**

Mount a local directory:

```
docker run \
    -v "$(pwd)":/app \
    my-app
```

This directly maps the current host directory to `/app` inside the container.

Bind mounts are especially useful during development because local code changes are immediately visible inside the container.

**Volume versus bind mount**

| | Volume | Bind Mount |
|---|---|---|
| Managed by Docker | Yes | No |
| Uses explicit host path | No | Yes |
| Good for persistent app data | Yes | Possible |
| Good for local development | Possible | Yes |

**.dockerignore**

A `.dockerignore` file prevents unnecessary files from being included in the build context.

Example:

```
.git
.venv
__pycache__
*.pyc
.env
data/
models/
notebooks/
```

This can:

- reduce build time;
- reduce accidental inclusion of large files;
- prevent secrets from being copied into the image;
- keep images cleaner.

Do not rely on `.dockerignore` as the only mechanism protecting secrets, but use it as an additional safeguard.

**Build cache**

Docker caches build layers.

This Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir \
    -r requirements.txt

COPY . .
```

is preferable to:

```
FROM python:3.12-slim

WORKDIR /app

COPY . .

RUN pip install --no-cache-dir \
    -r requirements.txt
```

because application code changes frequently while `requirements.txt` changes less frequently.

With the first structure, Docker can often reuse the dependency-installation layer.

A useful rule is:

> Put relatively stable build steps before frequently changing build steps.

**Keeping images small**

Useful practices include:

- use an appropriately small base image;
- exclude unnecessary files with `.dockerignore`;
- avoid unnecessary system packages;
- remove temporary installation files;
- use `pip install -no-cache-dir`;
- use multi-stage builds when build dependencies are not needed at runtime.

Inspect image size:

```
docker images
```

**Multi-stage builds**

Multi-stage builds separate build dependencies from the final runtime image.

General structure:

```
FROM some-image AS builder

# Build application here


FROM smaller-runtime-image

# Copy only required output
```

```
COPY --from=builder /build/output /app

CMD ["..."]
```

This is particularly useful for compiled applications or frontend builds where the compiler/build toolchain is unnecessary at runtime.

### Docker Compose

Docker Compose is useful when an application contains multiple services.

For example:

```
Web Application
+ Database
+ Cache
```

A simple `compose.yaml`:

```
services:

  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_HOST: db
    depends_on:
      - db

  db:
    image: postgres
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

Start all services:

```
docker compose up
```

Run in background:

```
docker compose up -d
```

Build/rebuild:

```
docker compose up --build
```

Stop and remove containers/networks:

```
docker compose down
```

View logs:

```
docker compose logs
```

Follow logs:

```
docker compose logs -f
```

List services:

```
docker compose ps
```

### Container networking

A common source of confusion is:

```
localhost
```

Inside a container, `localhost` refers to that container itself.

If a Compose service named `app` needs to communicate with a service named `db`, it can typically use the service name:

```
db
```

rather than:

```
localhost
```

For example:

```
DATABASE_HOST=db
```

This is one of the most common Docker networking issues during development.

### Typical development workflow

Build:

```
docker build -t my-app .
```

Run:

```
docker run \
    --name my-app-container \
    -p 8000:8000 \
    my-app
```

Check:

```
docker ps
```

Inspect logs:

```
docker logs -f my-app-container
```

Enter the container if necessary:

```
docker exec -it my-app-container sh
```

Stop:

```
docker stop my-app-container
```

Remove:

```
docker rm my-app-container
```

**Cleanup**

Remove stopped containers:

```
docker container prune
```

Remove unused images:

```
docker image prune
```

Remove unused Docker objects:

```
docker system prune
```

Check disk usage:

```
docker system df
```

Use cleanup commands carefully, especially when volumes or cached images are important.

**Containers versus virtual machines**

| | Container | Virtual Machine |
|---|---|---|
| Isolation | Process-level | Full OS-level |
| Guest OS | Shares host kernel | Separate guest OS |
| Startup | Usually fast | Usually slower |
| Size | Usually smaller | Usually larger |
| Portability | High | High, but heavier |

Containers are not simply lightweight virtual machines. They isolate processes while sharing the host operating system kernel.

**Common failure modes**

**Container immediately exits**

Check:

```
docker ps -a
docker logs container_name
```

A container normally runs only while its main process is running.

**Application works locally but cannot be reached**

Check:

- whether the correct port was published;
- whether the application is listening on the expected container port;
- whether the application is bound only to `127.0.0.1` inside the container.

Web applications in containers commonly need to listen on:

```
0.0.0.0
```

rather than only:

```
127.0.0.1
```

**Code changes do not appear**

Possible causes:

- the image was not rebuilt;
- Docker reused a cached layer;
- the wrong image/tag is running;
- a bind mount is overriding files inside the image.

Rebuild when needed:

```
docker build -t my-app .
```

Force a build without cache when debugging cache issues:

```
docker build --no-cache -t my-app .
```

### Container cannot reach another service

Remember that:

```
localhost
```

inside a container refers to that container, not another container and not automatically the host machine.

### Practical tips

- Remember the sequence: Dockerfile → image → container.
- Use meaningful image tags instead of relying only on `latest`.
- Keep images small and dependencies explicit.
- Use `.dockerignore` to reduce unnecessary build context.
- Copy dependency files before application code to improve build caching.
- Store persistent data outside the container filesystem.
- Use environment variables for runtime configuration.
- Never bake credentials or private keys into an image.
- Use `docker logs` first when a container fails.
- Use `docker exec` when you need to inspect a running container.
- Remember that `EXPOSE` does not publish a port.
- Remember that container `localhost` refers to the container itself.
- Bind web applications to `0.0.0.0` when they need to be reachable outside the container.
- Use Compose when several services must run together.
- Pin important dependency and base-image versions for reproducibility.

### Interview notes

Be comfortable explaining the distinction between:

- Dockerfile, image, and container;
- `RUN` and `CMD`;

- build time and runtime;
- image and container;
- volume and bind mount;
- container port and host port;
- containers and virtual machines;
- Docker and Docker Compose.

Common interview questions include:

- **Why use Docker?**
  To package an application and its dependencies into a reproducible, portable runtime environment.
- **What is the difference between an image and a container?**
  An image is an immutable template; a container is a running instance of that image.
- **What happens to data when a container is deleted?**
  Data stored only in the container's writable filesystem can disappear. Persistent data should be stored in volumes or external systems.
- **What does `-p 8080:8000` mean?**
  Host port 8080 is forwarded to port 8000 inside the container.
- **Why does Docker build caching matter?**
  Docker can reuse unchanged layers, so ordering Dockerfile instructions carefully can make repeated builds substantially faster.
- **Why might an application work outside Docker but fail inside it?**
  Common causes include missing dependencies, incorrect environment variables, incorrect port mappings, filesystem assumptions, networking differences, or binding the application only to `localhost`.
- **When would you use Docker Compose?**
  When an application requires multiple coordinated services such as a web application, database, worker, or cache.

For technical interviews, knowing how to debug a broken container is often more useful than memorizing Docker commands. A strong first sequence is:

```
docker ps -a
docker logs container_name
docker inspect container_name
docker exec -it container_name sh
```

This answers four useful questions: whether the container exists and is running, what it logged, how it is configured, and what its internal environment looks like.

# 6 Reporting

## 6.1 Power BI

**Power BI** is Microsoft's business intelligence platform for transforming data into interactive reports and dashboards.

The core workflow is:

```
Data Source
-> Power Query
-> Data Model
-> DAX
-> Visualizations
-> Report
```

The most important skills for day-to-day Power BI work are:

- cleaning data with Power Query;
- designing a good relational model;
- creating measures with DAX;
- understanding filter context;
- building useful rather than merely attractive reports.

**Core terminology**

| Term | Meaning |
|---|---|
| Power BI Desktop | Main application for building models and reports |
| Power Query | Data import and transformation layer |
| Semantic Model | Tables, relationships, measures, and business logic |
| DAX | Formula language used for calculations |
| Measure | Calculation evaluated according to filter context |
| Calculated Column | Value calculated and stored for each row |
| Report | Multi-page interactive collection of visuals |
| Dashboard | Service-level canvas containing pinned report visuals |
| Slicer | Interactive visual used to filter report data |

**Power Query**

Power Query is used to load and transform data before it enters the analytical model.

Common transformations include:

- changing data types;
- renaming columns;

- filtering rows;
- removing duplicates;
- splitting or combining columns;
- joining datasets;
- appending datasets;
- pivoting and unpivoting;
- handling missing values.

A useful rule is:

> Use Power Query for data preparation and DAX for analytical calculations.

Do not use complicated DAX to compensate for data that should have been cleaned or reshaped before entering the model.

### Applied Steps

Power Query records transformations as ordered steps.

Conceptually:

```
Source
-> Select Columns
-> Change Types
-> Filter Rows
-> Remove Duplicates
-> Merge Tables
-> Load
```

Order matters because each step operates on the result of the previous step.

### Merge versus Append

- **Merge**: combine tables horizontally using matching keys.
- **Append**: combine tables vertically by stacking rows.

Think:

```
Merge  -> SQL JOIN
Append -> SQL UNION ALL
```

### Data modeling

A well-designed model is usually more important than complicated DAX.

Power BI works particularly well with a **star schema**.

A typical model contains:

```
          DimDate
             |
DimCustomer--FactSales--DimProduct
             |
          DimStore
```

### Fact tables

Fact tables contain observations or events such as:

- sales transactions;
- claims;
- orders;
- visits;
- measurements.

They commonly contain:

- foreign keys;
- dates;
- quantities;
- monetary values;
- measurable outcomes.

### Dimension tables

Dimension tables describe entities used to group or filter facts.

Examples:

- customer;
- product;
- date;
- geography;
- department.

### Grain

Before modeling a table, determine:

> What does one row represent?

For example:

```
FactSales:
one row = one transaction line

DimCustomer:
one row = one customer
```

Mixing different grains is a common source of incorrect calculations.

### Relationships

A common star-schema relationship is:

```
Dimension (1) -> (*) Fact
```

For example:

```
DimCustomer[CustomerID]
        |
        | 1:*
        |
FactSales[CustomerID]
```

Understand:

- one-to-one;
- one-to-many;
- many-to-many;
- active versus inactive relationships;
- filter direction.

Many-to-many and bidirectional relationships can be useful, but they should not be the default solution to a poorly designed model.

### Practical rule

Prefer simple:

```
Dimension -> Fact
```

filtering whenever the model permits it.

### DAX

DAX (Data Analysis Expressions) is Power BI's analytical formula language.

A basic measure:

```
Total Sales =
SUM(Sales[Revenue])
```

Average:

```
Average Sale =
AVERAGE(Sales[Revenue])
```

Count rows:

```
Number of Sales =
COUNTROWS(Sales)
```

Distinct count:

```
Customers =
DISTINCTCOUNT(Sales[CustomerID])
```

### Measures versus calculated columns

This distinction is fundamental.

| | Measure | Calculated Column |
|---|---|---|
| Computed | At query/report time | For each row |
| Depends on filters | Yes | Not in the same way |
| Stored per row | No | Yes |
| Typical use | Aggregation/KPI | Row-level derived value |

Calculated column:

```
Profit =
Sales[Revenue] - Sales[Cost]
```

Measure:

```
Total Profit =
SUM(Sales[Revenue]) -
SUM(Sales[Cost])
```

For report-level aggregations, measures are generally preferable.

### Filter context

Filter context is one of the most important concepts in DAX.

Suppose:

```
Total Sales =
SUM(Sales[Revenue])
```

The same measure may display different values when the report is filtered by:

- year;
- product;
- region;
- customer;
- slicer selections.

The formula does not change. The **filter context** changes.

This is why measures dynamically respond to report interactions.

### CALCULATE

`CALCULATE` evaluates an expression under modified filter context.

Example:

```
US Sales =
CALCULATE(
    [Total Sales],
    Customer[Country] = "USA"
)
```

Another example:

```
Sales 2026 =
CALCULATE(
    [Total Sales],
    DimDate[Year] = 2026
)
```

A useful way to remember it is:

> `CALCULATE` changes the filters under which an expression is evaluated.

This is one of the most important DAX concepts for interviews.

### DIVIDE

Instead of:

```
Profit Margin =
[Total Profit] / [Total Sales]
```

prefer:

```
Profit Margin =
DIVIDE(
```

```
    [Total Profit],
    [Total Sales]
)
```

`DIVIDE` handles zero denominators more safely.

### Removing filters

Example percentage of total:

```
Percent of Total =
DIVIDE(
    [Total Sales],
    CALCULATE(
        [Total Sales],
        REMOVEFILTERS(Product)
    )
)
```

The denominator removes the current product filter while retaining other relevant context.

### Iterator functions

Functions ending in `X` commonly iterate over a table.

Examples:

```
SUMX
AVERAGEX
MINX
MAXX
```

Example:

```
Total Profit =
SUMX(
    Sales,
    Sales[Quantity] *
    (Sales[Price] - Sales[UnitCost])
)
```

`SUM` aggregates an existing column.

`SUMX` evaluates an expression row-by-row and then sums the results.

### Date tables

Time-based reporting should generally use a dedicated date dimension.

Useful fields include:

```
Date
Year
Quarter
Month
Month Number
Week
Day
```

Connect the date table to the fact table using the appropriate date key.

A dedicated date table makes time-based filtering and comparisons much easier to manage consistently.

**Common visuals**

| Visual | Good Use |
|---|---|
| Card | Single KPI |
| Bar Chart | Compare categories |
| Line Chart | Time series/trends |
| Scatter Plot | Relationship between numeric variables |
| Table | Detailed values |
| Matrix | Hierarchical/tabular summaries |
| Map | Geographic patterns |
| Slicer | Interactive filtering |

Choose the visual based on the analytical question rather than visual novelty.

**Report interactions**

Power BI visuals can interact with one another through filtering and highlighting.

A user selecting a category in one chart may change several other visuals on the page.

Always test these interactions. A report can display individually correct charts while producing confusing behavior when selections propagate across the page.

**Import versus DirectQuery**

| | Import | DirectQuery |
|---|---|---|
| Data location | Loaded into Power BI | Remains in source |
| Typical performance | Faster | Source dependent |
| Refresh needed | Yes | Queries source |
| Model flexibility | High | More constrained |
| Source load | Lower during viewing | Queries source during use |

Import mode is often preferable when the dataset fits comfortably and scheduled refreshes are acceptable.

DirectQuery is useful when querying the underlying source is necessary, but report performance becomes more dependent on source-system performance and model design.

**Performance tips**

- Prefer a clean star schema.
- Remove unused columns before loading large datasets.
- Avoid unnecessary high-cardinality text columns.
- Prefer measures for report-level calculations.
- Avoid excessive bidirectional relationships.
- Avoid unnecessarily complicated DAX when the model can be simplified.
- Reduce the number of unnecessary visuals on a page.
- Push appropriate transformations upstream when practical.

**Practical tips**

- Determine table grain before creating relationships.
- Build the model before spending significant time formatting visuals.
- Prefer star schemas over one giant flattened table when appropriate.
- Use Power Query for data preparation and DAX for analytical logic.
- Prefer measures over calculated columns for dynamic aggregations.
- Learn `CALCULATE`; it is central to serious DAX work.
- Create a proper date table for time-based reporting.
- Test slicers and cross-filtering interactions.
- Use clear business names for measures.
- Validate important dashboard totals against the source data.
- Avoid putting every available metric on one page.
- Design each report page around a small number of analytical questions.

**Interview notes**

Be comfortable explaining:

- Power Query versus DAX;
- fact tables versus dimension tables;
- star schemas;

- one-to-many relationships;
- measures versus calculated columns;
- row context versus filter context;
- the purpose of `CALCULATE`;
- Import versus DirectQuery;
- why model design affects report performance.

A common interview question is:

> Why is my Power BI measure returning different values in different visuals?

A strong first answer is to investigate **filter context**: slicers, visual-level filters, page filters, relationships, and the dimensions used by each visual can change the context in which the measure is evaluated.

## 6.2 Tableau

**Tableau** is a visual analytics and business intelligence platform used to explore data and build interactive dashboards.

The basic workflow is:

```
Connect to Data
-> Define Fields
-> Build Worksheets
-> Add Filters / Calculations
-> Combine into Dashboard
```

Tableau is especially centered around interactive visual exploration and the relationship between **dimensions**, **measures**, and **level of detail**.

**Core terminology**

| Term | Meaning |
|---|---|
| Worksheet | Individual visualization |
| Dashboard | Collection of worksheets and interactive objects |
| Story | Sequence of sheets/dashboards used to communicate a narrative |
| Dimension | Field generally used to categorize or segment data |
| Measure | Field generally used for quantitative aggregation |
| Discrete | Produces distinct headers/categories |
| Continuous | Produces a continuous axis |
| Calculated Field | Derived value defined using a Tableau expression |
| Parameter | User-controlled value usable in calculations and interactions |

### Dimensions versus measures

Examples of dimensions:

```
Customer
Region
Product
Category
Date
```

Examples of measures:

```
Sales
Profit
Quantity
Cost
```

A simple way to think about them is:

- **Dimension**: how should the data be grouped?
- **Measure**: what should be measured or aggregated?

For example:

```
Sales by Region
```

uses:

```
Region -> Dimension
Sales  -> Measure
```

### Discrete versus continuous

Tableau also distinguishes between discrete and continuous fields.

- **Discrete**: creates headers or categories.
- **Continuous**: creates an axis over a continuous range.

This distinction is separate from dimension versus measure.

A date, for example, can be displayed as either discrete categories or a continuous timeline depending on the desired visualization.

### Rows and Columns shelves

A simple bar chart might use:

```
Columns: Region
Rows:    SUM(Sales)
```

A time series might use:

```
Columns: Date
Rows:    SUM(Sales)
```

Dragging fields between Rows and Columns changes the structure of the visualization.

**Marks card**

The Marks card controls how observations are represented.

Common properties include:

- Color;
- Size;
- Label;
- Detail;
- Tooltip;
- Shape.

For example:

```
Columns: Sales
Rows:    Profit
Color:   Region
Detail:  Customer
```

creates a scatter plot where customer observations are colored by region.

**Aggregation**

Measures are usually aggregated.

Common aggregations include:

```
SUM(Sales)
AVG(Sales)
MIN(Sales)
MAX(Sales)
COUNT(Sales)
COUNTD(CustomerID)
```

`COUNTD` means distinct count.

Always check the aggregation being used. A visualization of:

```
SUM(Profit)
```

answers a different question from:

```
AVG(Profit)
```

### Calculated fields

Basic arithmetic:

```
[Sales] - [Cost]
```

Conditional logic:

```
IF [Profit] > 0 THEN
    "Profitable"
ELSE
    "Unprofitable"
END
```

Multiple conditions:

```
IF [Sales] >= 10000 THEN
    "High"
ELSEIF [Sales] >= 5000 THEN
    "Medium"
ELSE
    "Low"
END
```

Handle missing values:

```
IFNULL([Profit], 0)
```

### Table calculations

Table calculations operate over values already present in the visualization.

Examples include:

- running totals;
- percent of total;
- rank;
- moving averages;
- differences from previous values.

Running total:

```
RUNNING_SUM(SUM([Sales]))
```

Rank:

```
RANK(SUM([Sales]))
```

Previous value:

```
LOOKUP(SUM([Sales]), -1)
```

Table calculations depend heavily on how the visualization is partitioned and addressed. If a table calculation produces unexpected results, inspect the direction and level over which it is computing.

**Level of Detail (LOD) expressions**

LOD expressions explicitly control the granularity at which a calculation is performed.

**FIXED**

Calculate customer-level sales regardless of the dimensions currently shown in the view:

```
{ FIXED [Customer ID] :
    SUM([Sales])
}
```

**INCLUDE**

Include an additional dimension in the calculation:

```
{ INCLUDE [Customer ID] :
    SUM([Sales])
}
```

**EXCLUDE**

Remove a dimension from the calculation:

```
{ EXCLUDE [Region] :
    SUM([Sales])
}
```

The three major LOD expressions are:

- `FIXED`: calculate at an explicitly specified level.
- `INCLUDE`: add dimensions to the calculation's level.
- `EXCLUDE`: remove dimensions from the calculation's level.

LOD expressions are one of the most important Tableau concepts for technical interviews.

**Filters**

Common filters include:

- data source filters;
- extract filters;
- context filters;
- dimension filters;
- measure filters;
- table-calculation filters.

Filtering order can matter, especially when LOD expressions or Top-N filters are involved.

If a calculation produces an unexpected result, consider whether the relevant filter is applied before or after the calculation.

**Context filters**

A context filter creates a subset of data that other filters can operate on.

This is particularly useful when a secondary filter should be evaluated only within an already filtered subset.

For example:

```
Context Filter: Region = East
Top-N Filter:   Top 10 Customers
```

can mean:

```
Top 10 customers within the East region
```

rather than top customers globally followed by an East-region filter.

**Parameters**

Parameters are user-controlled values that can drive calculations.

For example, create a parameter:

```
Top N
```

and use it in a calculated field:

```
RANK(SUM([Sales])) <= [Top N]
```

Parameters can support:

- Top-N controls;
- thresholds;
- scenario analysis;

- metric selection;
- reference values.

**Relationships versus joins**

Tableau can combine tables using relationships or physical joins.

A physical join combines rows before visualization:

```
Orders
LEFT JOIN
Customers
```

Relationships allow tables to remain logically distinct and can preserve their individual levels of detail until they are needed in a visualization.

As with SQL, always understand:

```
1:1
1:N
N:1
N:M
```

before combining tables.

Unexpected row multiplication can produce incorrect measures.

**Live connections versus extracts**

| | Live | Extract |
|---|---|---|
| Data location | Source system | Tableau extract |
| Freshness | Current source data | Refresh dependent |
| Performance | Source dependent | Often faster |
| Offline use | Limited | Possible |
| Source load | Queries source | Reduced during analysis |

An extract is a stored analytical snapshot optimized for Tableau.

Use the approach appropriate for the required balance between freshness and performance.

**Dashboards**

Dashboards combine worksheets into an interactive reporting interface.

Useful dashboard components include:

- worksheets;
- filters;
- legends;

- parameters;
- text;
- navigation controls.

A useful dashboard should answer a clear analytical question rather than simply display as many charts as possible.

**Dashboard actions**

Actions allow one visualization to affect another.

Common actions include:

- filter actions;
- highlight actions;
- parameter actions;
- navigation actions.

For example, selecting a state on a map can filter a sales chart to that state.

Test interactions carefully so users understand what is currently filtering the dashboard.

**Common visual choices**

| Visual | Good Use |
|---|---|
| Bar Chart | Category comparisons |
| Line Chart | Trends over time |
| Scatter Plot | Relationships between numeric variables |
| Map | Geographic patterns |
| Histogram | Distribution of a numeric variable |
| Box Plot | Distribution and group comparison |
| Heat Map | Patterns across two categorical dimensions |
| Table | Detailed values |

Avoid choosing a visualization simply because it looks interesting. The visual should make the intended comparison easier to understand.

**Performance tips**

- Avoid loading unnecessary columns and rows.
- Reduce extremely high-cardinality dimensions when they are not needed.
- Avoid excessive marks in a single visualization.
- Use extracts when appropriate for performance.
- Avoid unnecessary complex calculations.

- Understand the grain of joined or related tables.
- Reduce unnecessary dashboard objects.

**Practical tips**

- Always check whether a field is a dimension or measure.
- Check whether a field is discrete or continuous.
- Verify the aggregation applied to every important measure.
- Understand the data grain before combining tables.
- Use LOD expressions when a calculation needs a specific granularity.
- Check table-calculation direction when results appear incorrect.
- Use parameters for user-controlled analytical behavior.
- Keep dashboards focused on a small number of questions.
- Test filter and dashboard actions before publishing.
- Validate important totals against the source data.
- Use informative titles that describe what a visual actually shows.

**Interview notes**

Be comfortable explaining:

- dimensions versus measures;
- discrete versus continuous fields;
- aggregation;
- calculated fields;
- table calculations;
- LOD expressions;
- `FIXED`, `INCLUDE`, and `EXCLUDE`;
- context filters;
- parameters;
- relationships versus joins;
- live connections versus extracts.

A common interview question is the difference between a table calculation and an LOD expression.

A useful answer is:

- A **table calculation** operates on the aggregated values present in the visualization and depends on the layout of the view.
- An **LOD expression** explicitly controls the granularity at which a calculation is performed relative to the visualization.

Another common question is when to use `FIXED`. Use it when a metric must be calculated at a specific level of detail regardless of some dimensions currently present in the view.

**Power BI versus Tableau**

| Concept | Power BI | Tableau |
|---|---|---|
| Data transformation | Power Query | Data Source / Prep workflow |
| Calculation language | DAX | Tableau calculations |
| Core model concept | Semantic/star model | Dimensions/measures and LOD |
| Reusable aggregation | Measure | Calculated field / LOD |
| Granularity control | Filter context / DAX | LOD expressions |
| User input | Parameters / slicers | Parameters / filters |
| Cached data | Import model | Extract |
| Source querying | DirectQuery | Live connection |
| Primary report unit | Report page | Worksheet/dashboard |

For interviews, avoid framing the difference as simply "Power BI is Microsoft and Tableau is Salesforce." The more useful technical distinction is how each tool approaches modeling and calculations.

In Power BI, spend particular attention on:

```
Data Model
Relationships
Measures
Filter Context
DAX
```

In Tableau, spend particular attention on:

```
Dimensions / Measures
Discrete / Continuous
Level of Detail
Table Calculations
Filters
```

For both tools, the most important reporting principle is the same:

> A visually polished dashboard is still wrong if the underlying data model, aggregation, filtering, or grain is wrong.